

CSC10006 – Hệ điều hành

Chương 2

QUẢN LÝ TIẾN TRÌNH – TIỂU TRÌNH



NỘI DUNG

- Quản lý tiến trình
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”

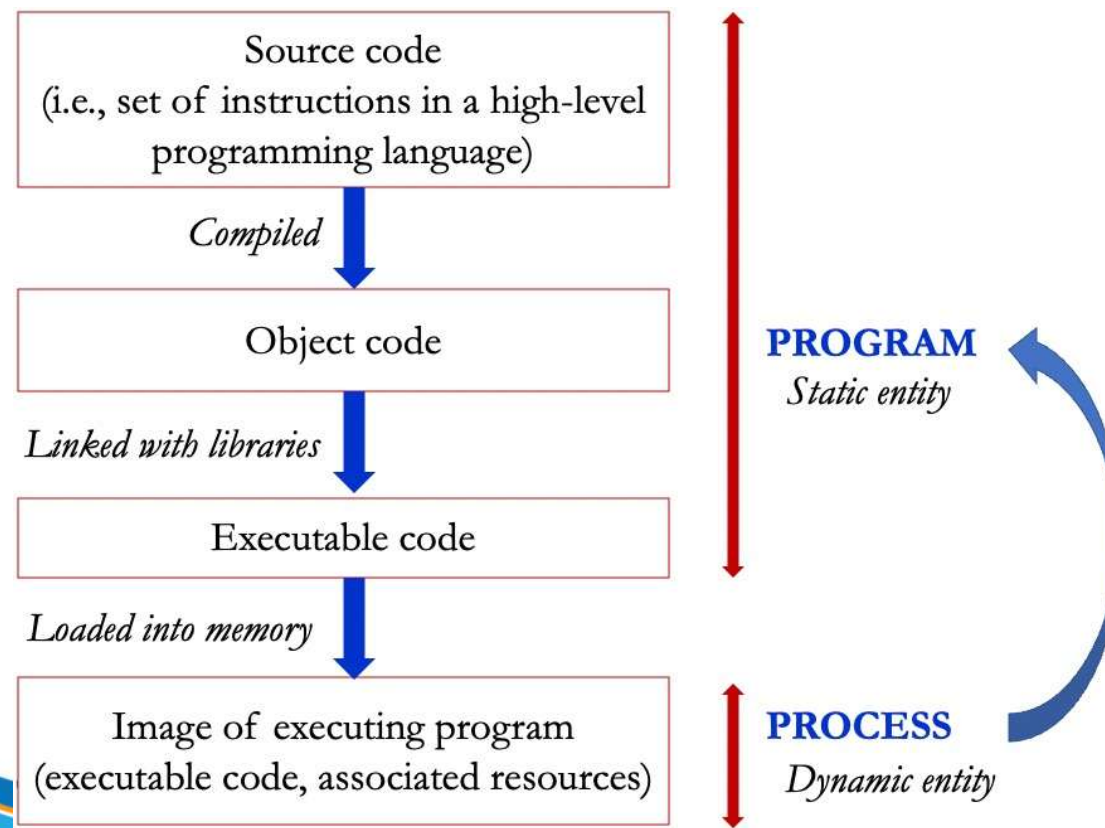


NỘI DUNG

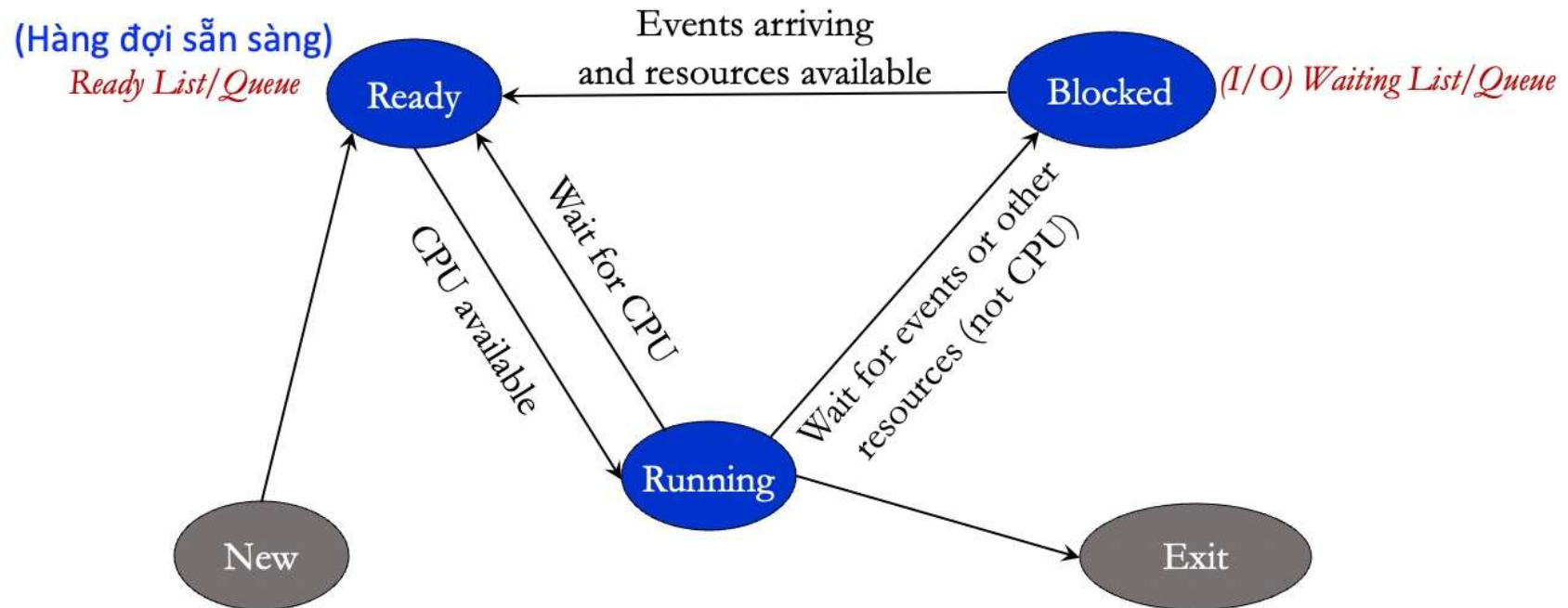
- Quản lý tiến trình
 - **Tổng quan về tiến trình**
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”

KHÁI NIỆM TIẾN TRÌNH

- Tiến trình (process): chương trình (program) đang thực thi, được cấp phát tài nguyên (CPU, bộ nhớ).



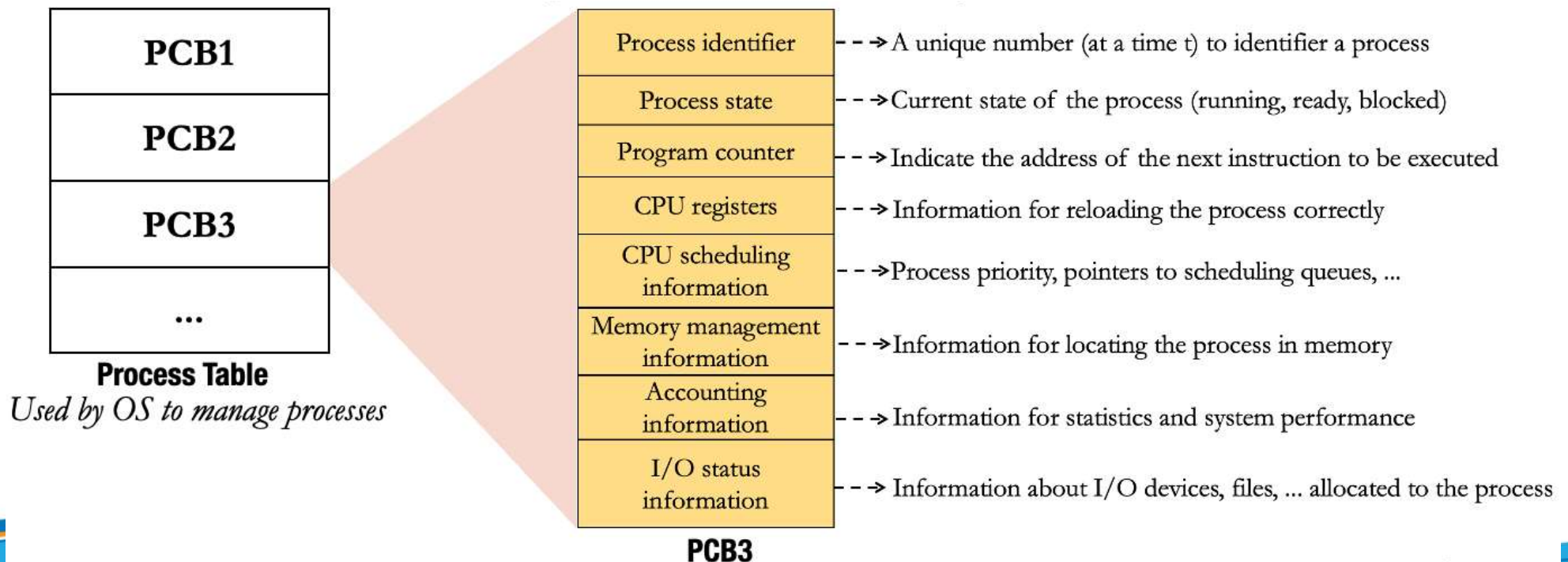
MÔ HÌNH TRẠNG THÁI TIẾN TRÌNH



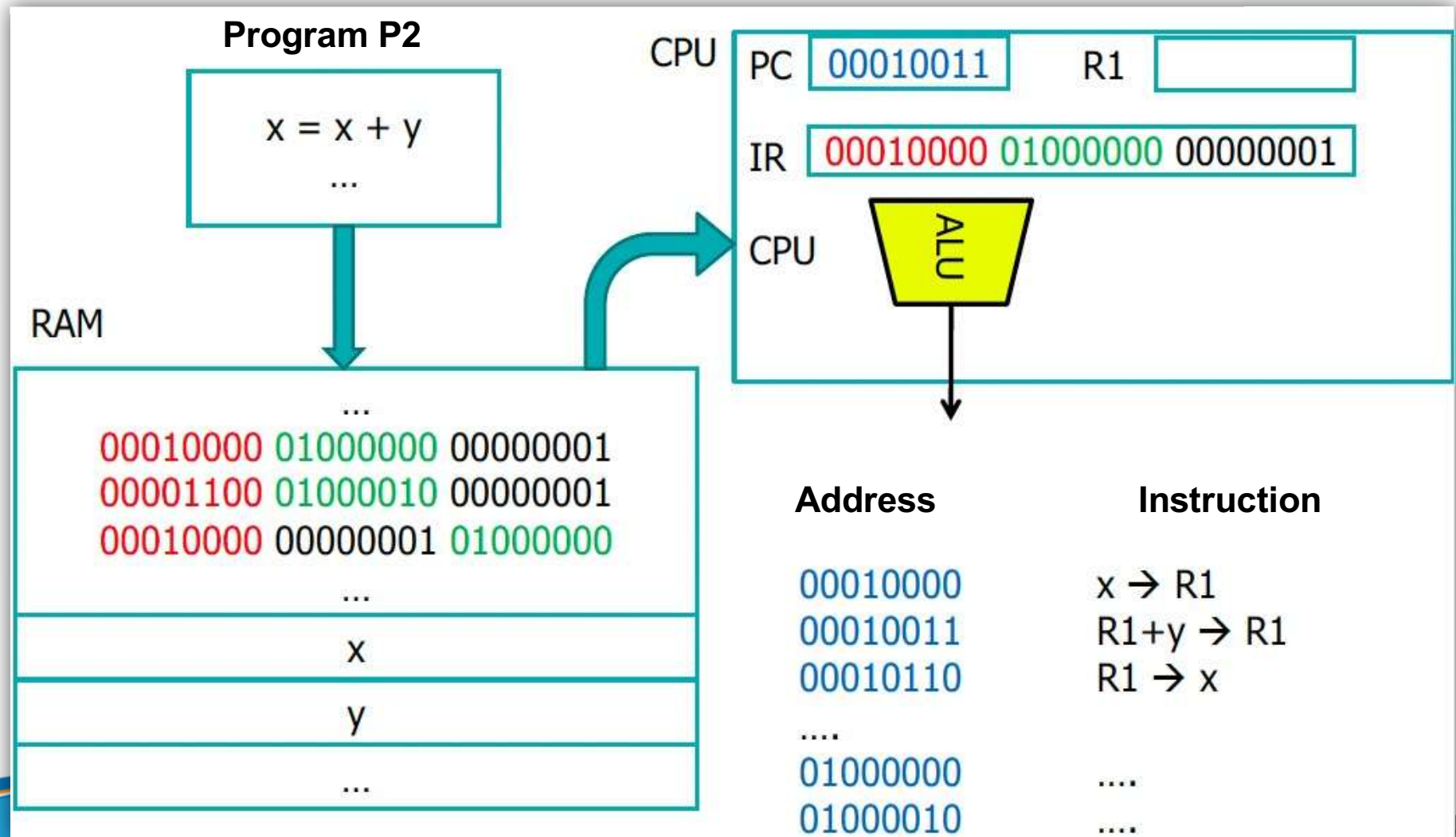
KHỐI QUẢN LÝ TIẾN TRÌNH (PCB)

(Process Control Block)

- Cấu trúc dữ liệu nằm trong nhân HĐH (Bảng tiến trình), lưu mọi thông tin cần thiết (định danh, trạng thái, ngữ cảnh, ...) để HĐH quản lý một tiến trình đang hoạt động.

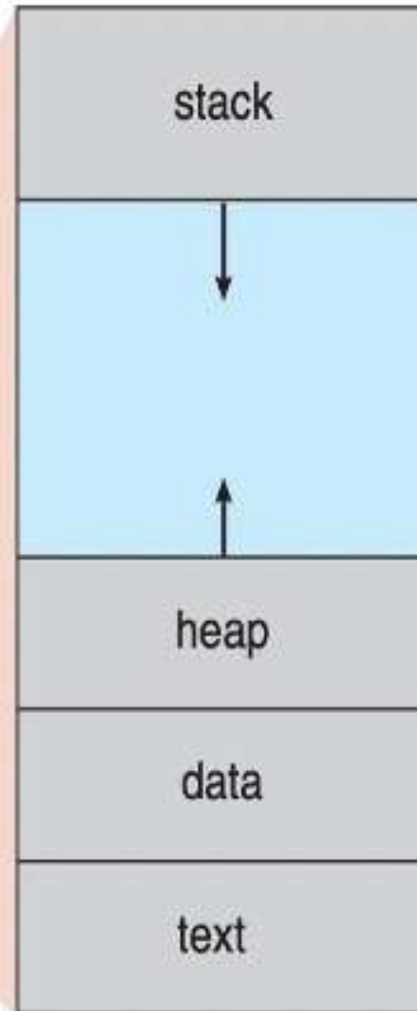
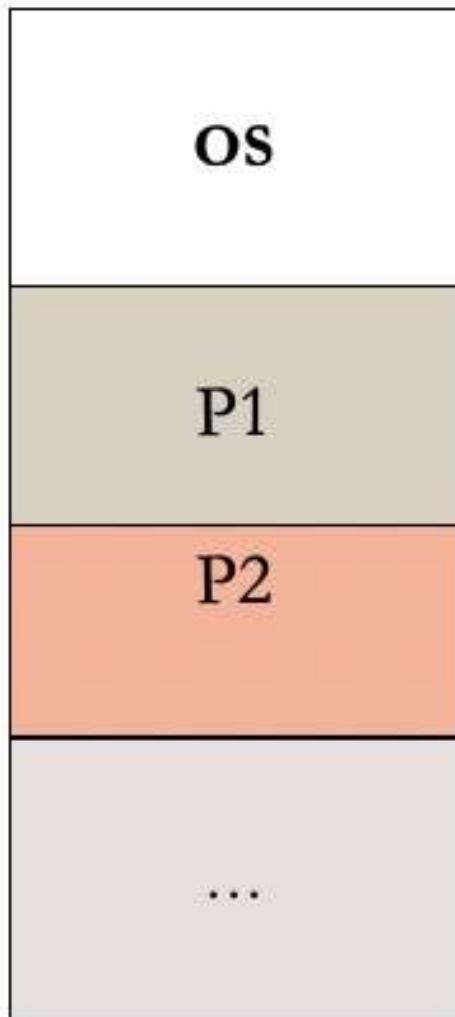


KHÔNG GIAN ĐỊA CHỈ TIẾN TRÌNH



KHÔNG GIAN ĐỊA CHỈ TIẾN TRÌNH (2)

RAM



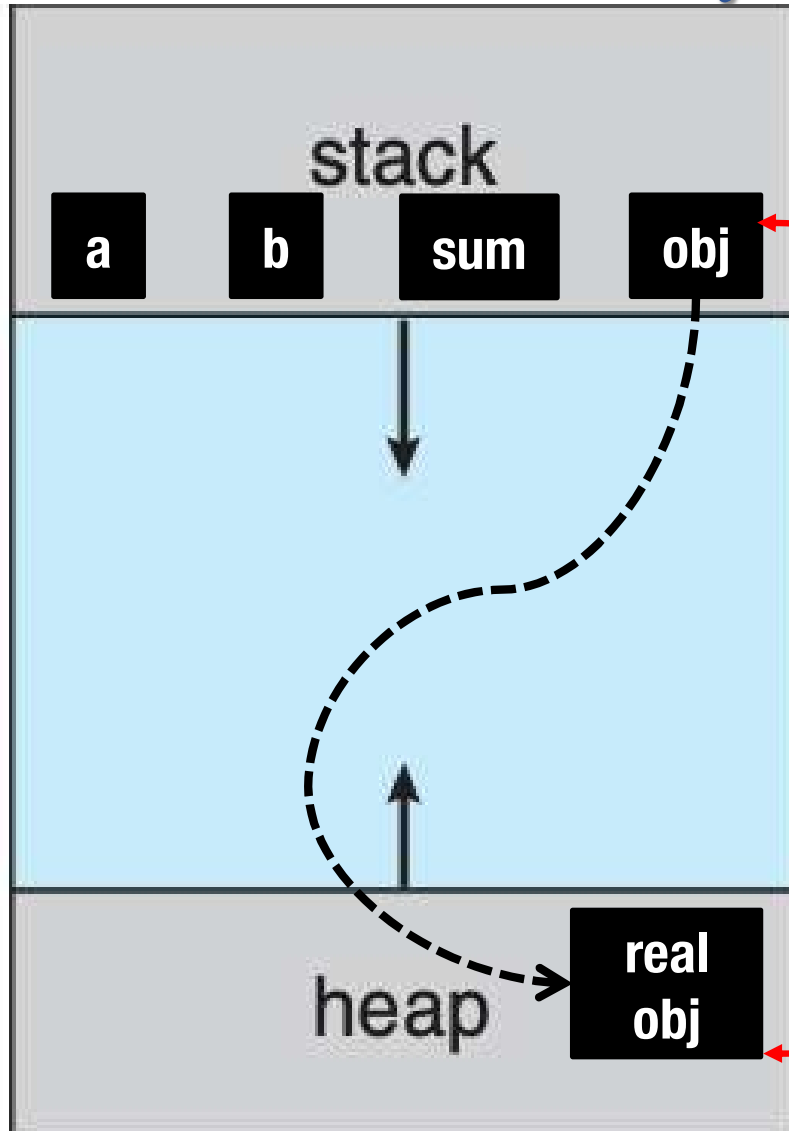
Biến cục bộ, tham số hàm (vùng nhớ được tự động cấp phát và thu hồi bởi hệ thống).

Biến cấp phát động (vùng nhớ được cấp phát và thu hồi theo yêu cầu của lập trình viên).

Biến toàn cục, biến tĩnh (vùng nhớ truy cập bởi mọi thủ tục).

Mã lệnh (instruction).

KHÔNG GIAN ĐỊA CHỈ TIẾN TRÌNH (3)



```
public class Addition {
    public int add(int a, int b) {
        int sum = a + b;
        return sum;
    }
}
```

```
void main() {
    Addition obj = new Addition();
    printf(obj→add(12, 25));
}
```

TẠO LẬP TIẾN TRÌNH

- Tiến trình cha (parent process) tạo tiến trình con (child process), và các tiến trình con có thể tiếp tục tạo con của chúng.
- Một tiến trình con có thể:
 - Là bản sao của tiến trình cha.
 - Tải một bản ảnh thực thi khác.
- Tiến trình mới sẽ được hệ thống cấp định danh (pid) và các tài nguyên liên quan, đồng thời PCB của tiến trình cũng được tạo và đưa vào bảng tiến trình.

TẠO LẬP TIỀN TRÌNH (2)

```
#include <unistd.h>
#include <stdio.h>
```

```
main(){
```

```
    int pid_t = fork();
```

```
    if ( pid_t == 0) { //child execution
```

```
        printf("%d is a child of parent %d", getpid(), getppid());
```

```
    }
```

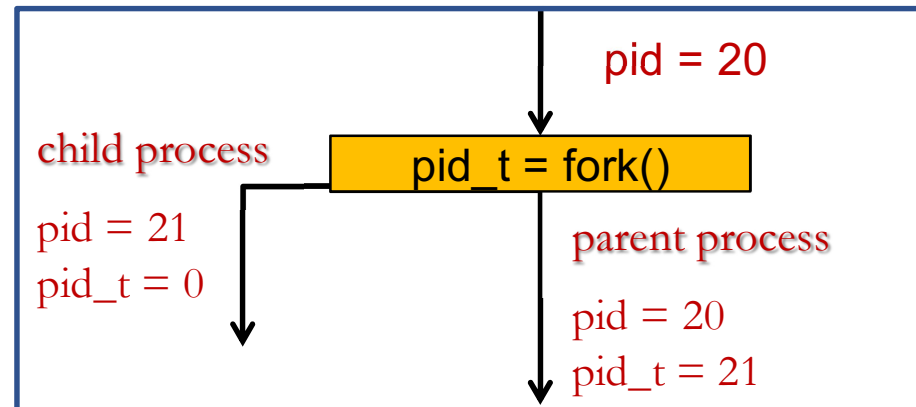
```
    else if ( pid_t > 0) { //parent execution
```

```
        printf("Parent %d created a child %d", getpid(), pid_t);
```

```
    }
```

```
    else { error }
```

```
}
```

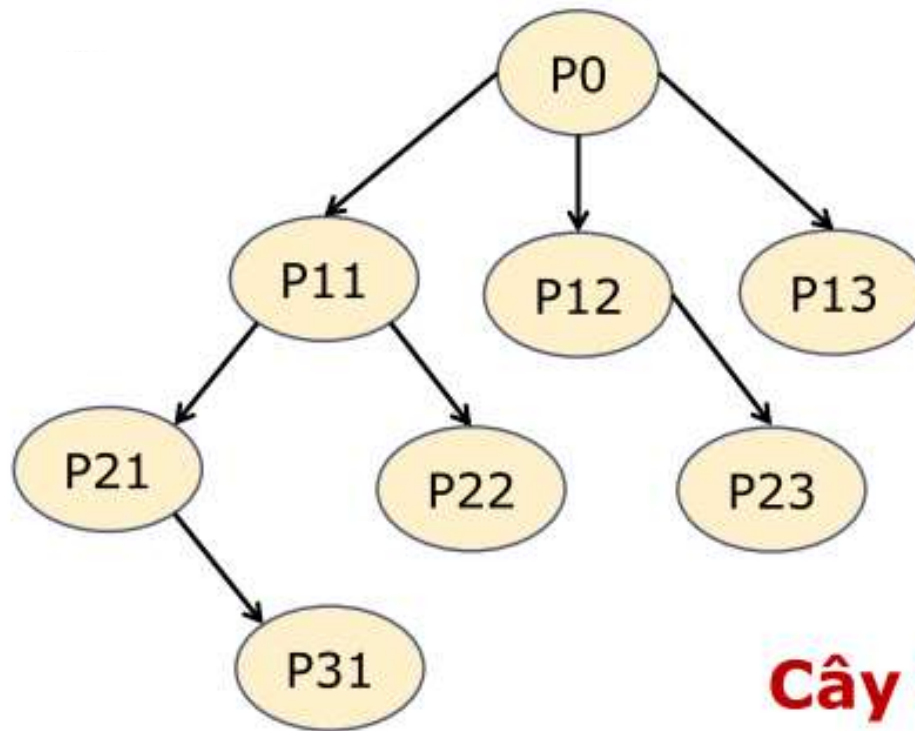


print 21 20

print 20 21

TẠO LẬP TIỀN TRÌNH (3)

```
for(int i = 0; i < 3; i++){ fork(); }
```



Cây tiến trình

HỦY TIẾN TRÌNH

- Tiến trình sẽ kết thúc khi hoàn thành công việc hoặc bị kết thúc bởi tiến trình cha hoặc hệ thống.
- Tiến trình “zombie”: tiến trình con kết thúc, nhưng PCB chưa được giải phóng hoàn toàn.
- **Thảo luận:** Chuyện gì xảy ra khi tiến trình cha kết thúc trước con?
 - Tiến trình “orphan”
 - Kết thúc dây chuyền (Cascading termination)

HỦY TIẾN TRÌNH (2)

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

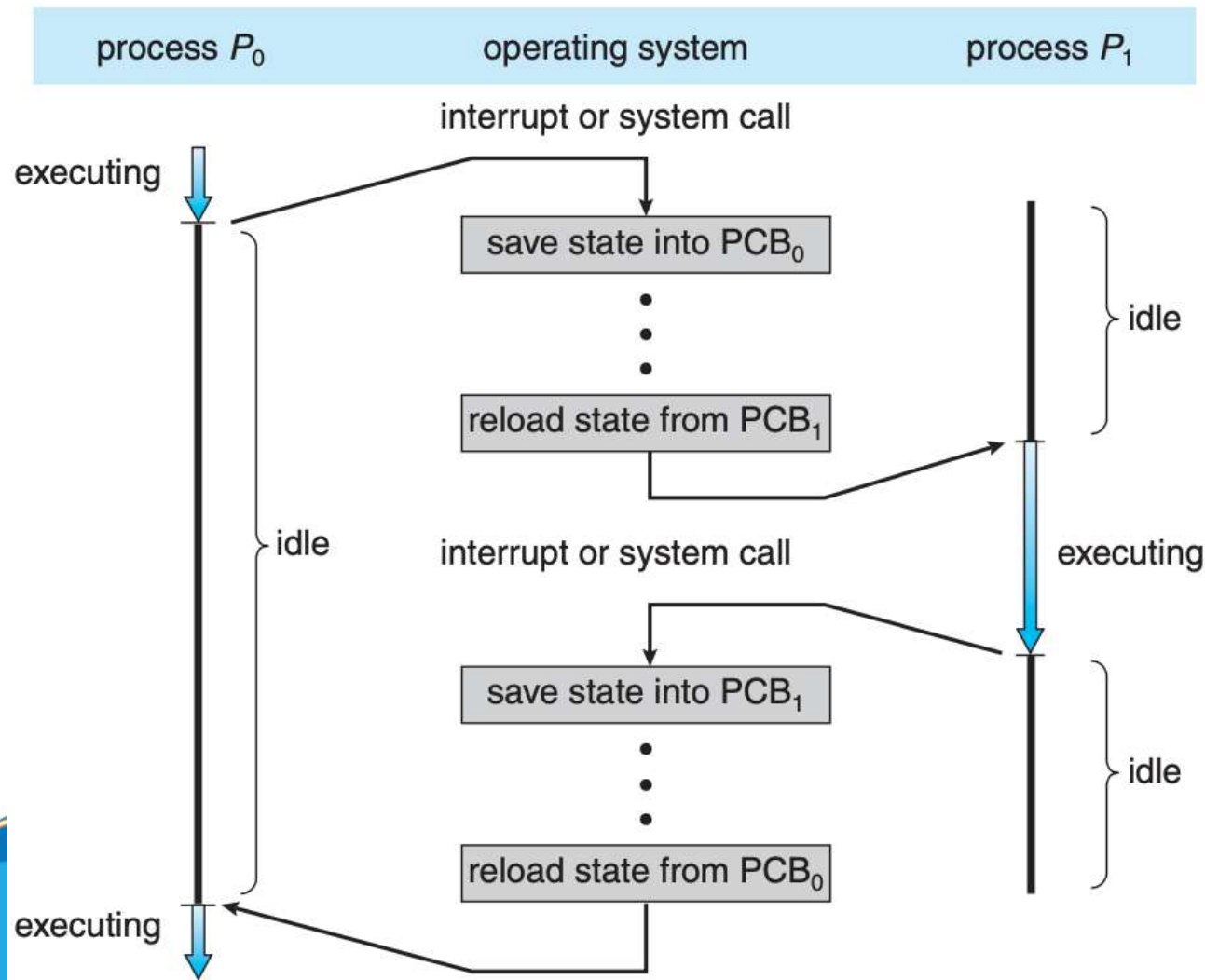
    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

TẠM DỪNG – TÁI THỰC THI TIẾN TRÌNH

CONTEXT SWITCH (Chuyển đổi ngữ cảnh)



NỘI DUNG

- Quản lý tiến trình
 - **Cơ chế liên lạc giữa các tiến trình (IPC)**
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”

LIÊN LẠC GIỮA CÁC TIẾN TRÌNH (IPC)

(Interprocess Communication)

Thảo luận: Tại sao các tiến trình cần cơ chế liên lạc?



Chia sẻ tài nguyên

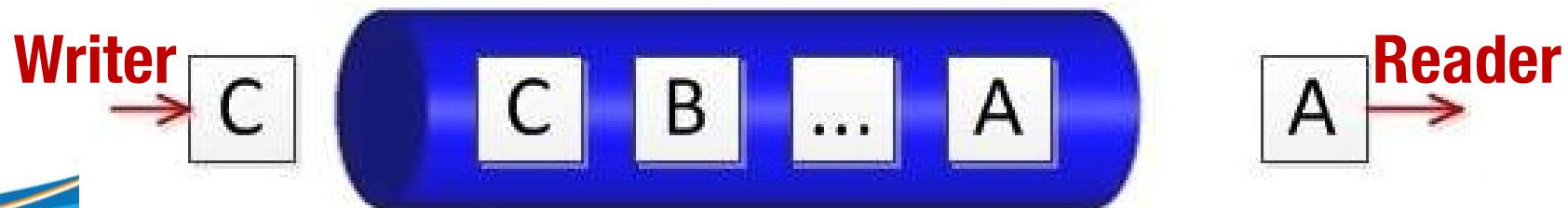


Phối hợp hoạt động

CƠ CHẾ LIÊN LẠC: ỐNG DẪN

(Pipe)

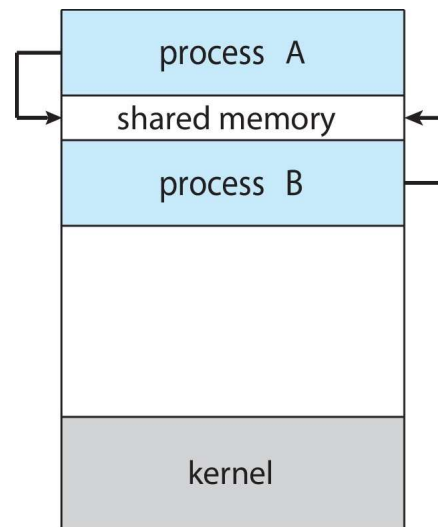
- Cơ chế liên lạc một chiều.
 - Tiến trình ghi (writer): chuyển dữ liệu vào ống.
 - Tiến trình đọc (reader): đọc dữ liệu từ ống.
- Quy định
 - Khi ghi, đóng đầu đọc.
 - Khi đọc, đóng đầu ghi.
- Phân loại: ống đặt tên; ống không đặt tên.



CƠ CHẾ LIÊN LẠC: VÙNG NHỚ CHIA SẺ

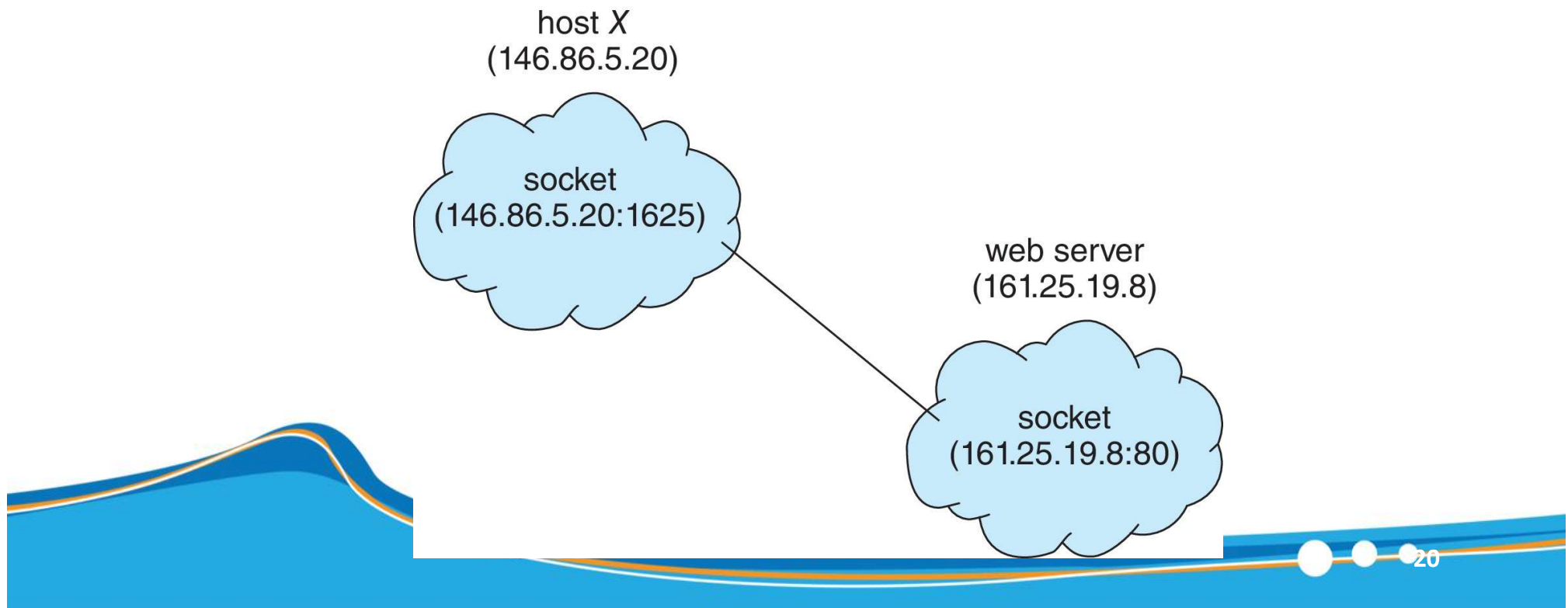
(Shared Memory)

- Cơ chế cho phép hai hay nhiều tiến trình chia sẻ “vùng nhớ dùng chung”.
- A thiết lập vùng nhớ chia sẻ trong không gian địa chỉ của A.
- B, C, ... “gắn” không gian dùng chung này vào không gian địa chỉ của nó.



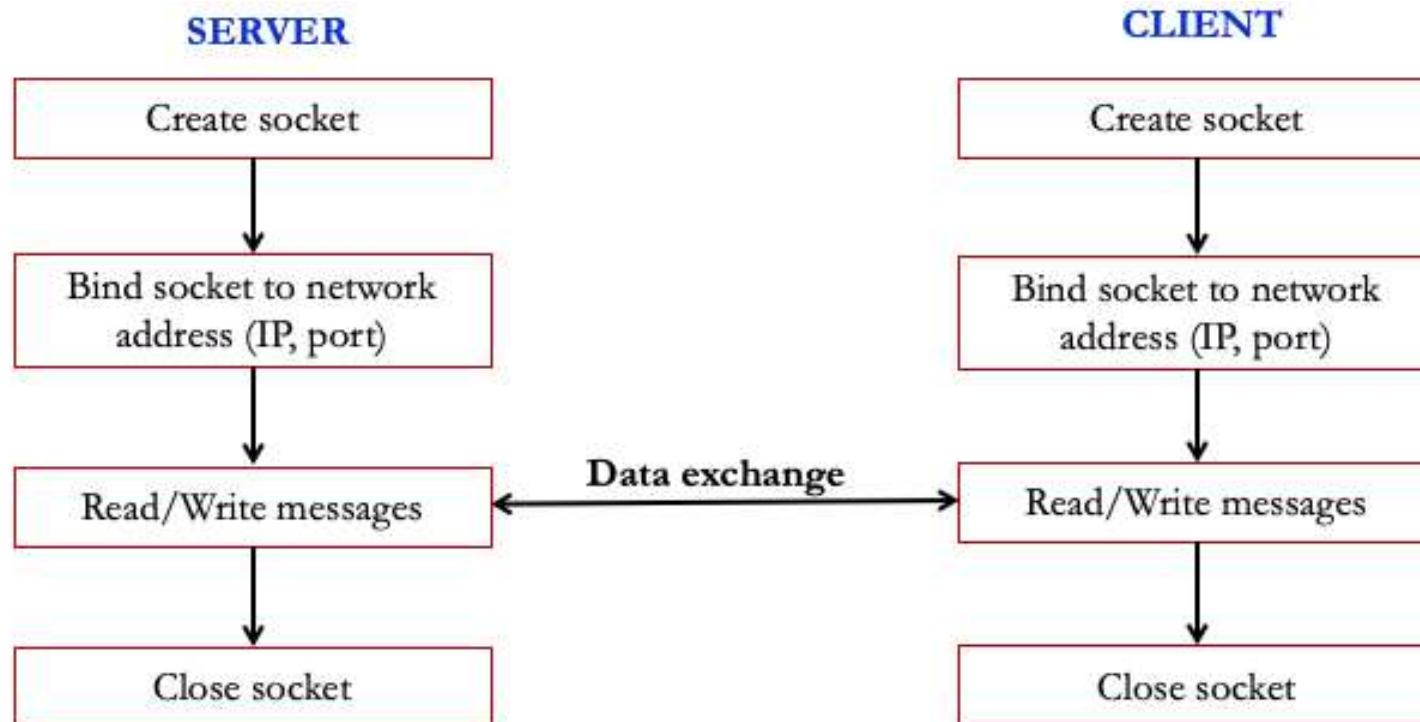
CƠ CHẾ LIÊN LẠC: SOCKET

- Cơ chế giao tiếp giữa các tiến trình ở các máy tính khác nhau, kết nối qua đường mạng.
- Cần địa chỉ định danh trên mạng (IP, Port).



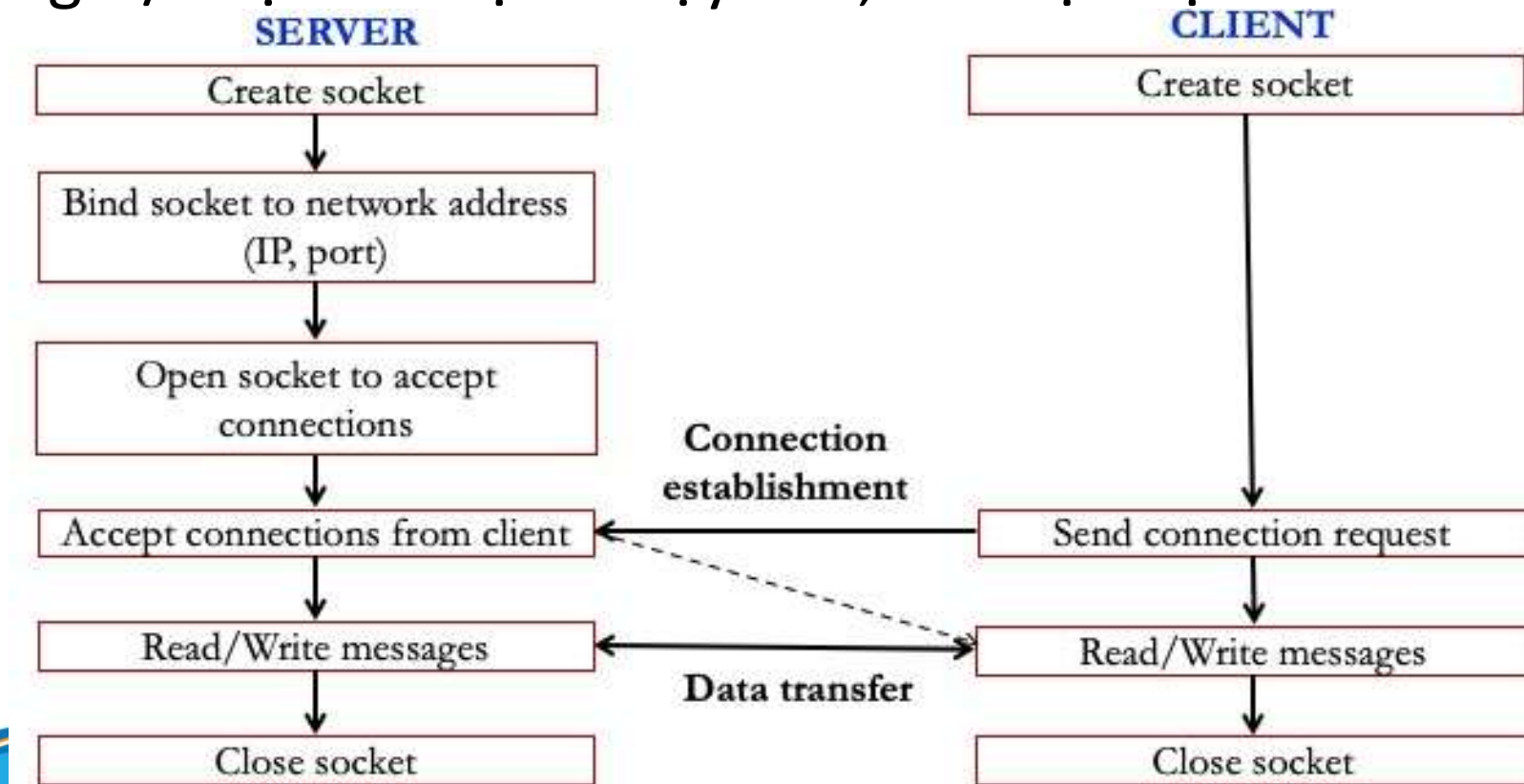
CƠ CHẾ LIÊN LẠC: SOCKET (2)

- UDP Socket: liên lạc không kết nối → Ưu tiên tốc độ nhưng độ tin cậy thấp.



CƠ CHẾ LIÊN LẠC: SOCKET (3)

- TCP Socket: thực hiện bắt tay ba bước trước khi gửi/nhận → độ tin cậy cao, tốc độ chậm hơn UDP.

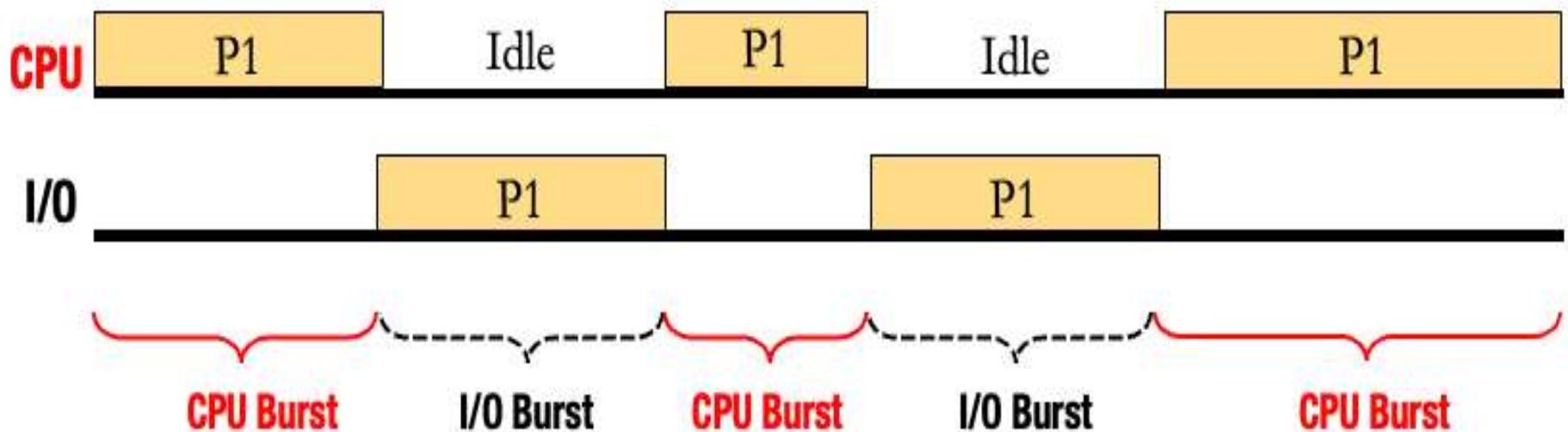


Nội dung

- Quản lý tiến trình
- Điều phối CPU
 - **Tổng quan về điều phối CPU (shortterm scheduling)**
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”

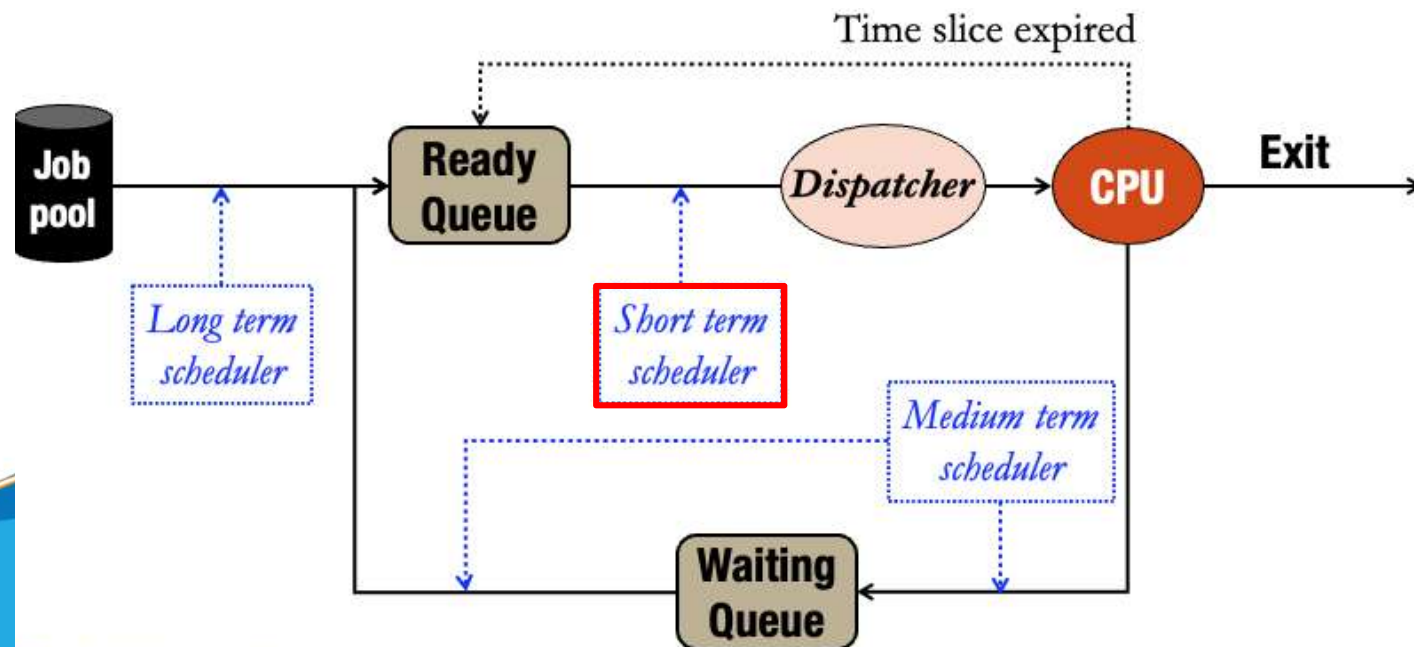
ĐIỀU PHỐI

- Quá trình xử lý của tiến trình luôn luân phiên giữa chu kỳ thực thi (CPU Burst) và nhập/xuất (I/O Burst).



ĐIỀU PHỐI (2)

- Bộ lập lịch (Shortterm Scheduler): lựa chọn tiến trình trong hàng đợi sẵn sàng (ready queue) để thực thi.
- Bộ điều phối (Dispatcher): thực hiện chuyển giao CPU cho tiến trình được chọn.



ĐIỀU PHỐI (2)

- Khi nào cần điều phối?
 - Có tiến trình mới được tạo lập.
 - Tiến trình đang thực thi kết thúc.
 - Tiến trình đang thực thi phải chờ đợi sự kiện.
 - Ngắt (interrupt) phát sinh (ngắt từ bộ định thời, từ thiết bị nhập/xuất, ...).

TIÊU CHÍ ĐIỀU PHỐI

- Công bằng và hiệu quả.
- Tối đa hoá hiệu suất CPU.
- Cực đại hóa thông lượng (throughput).
- Tối thiểu hoá:
 - Thời gian lưu trú (turnaround time).
 - Thời gian chờ (waiting time).
 - Thời gian hồi đáp (response time).



Nội dung

- Quản lý tiến trình
- Điều phối CPU
 - **Các thuật toán điều phối CPU**
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”

PHÂN LOẠI

- **Điều phối độc quyền** (non-preemptive scheduling): tiến trình tự nguyện trao trả CPU **khi kết thúc** hoặc bị khóa do **chờ đợi** tài nguyên hoặc sự kiện.
- **Điều phối không độc quyền** (preemptive scheduling): hệ thống có thể thu hồi CPU cho **tiến trình ưu tiên hơn** hoặc khi **hết lượng tử thời gian** (time slice, time quantum) cấp phát.



ĐIỀU PHỐI FCFS (First Come First Served)

- Độc quyền
- Tiến trình đến trước sẽ được thực thi trước
- Phù hợp hệ thống theo lô
- 😊 Đơn giản
- 😞 Hiệu suất không cao
- 😞 Chưa cân nhắc tiến trình cần ưu tiên



ĐIỀU PHỐI FCFS (First Come First Served)

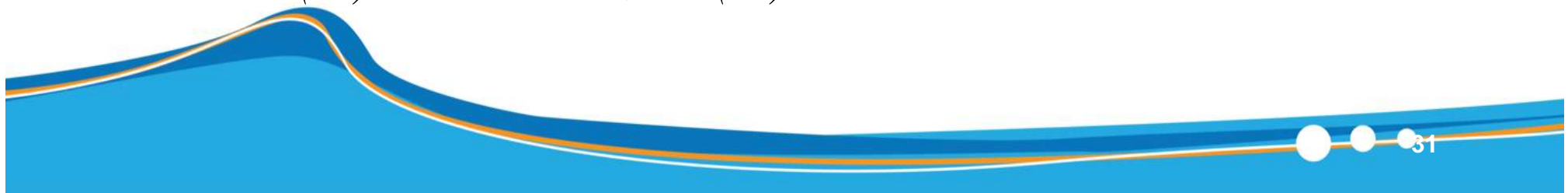
Tiến trình	Thời gian đến	TG thực thi	TT	WT
P1	0	24	24	0
P2	1	5	28	23
P3	2	3	30	27
			AVG_{TT} = 27.33	AVG_{WT} = 16.67



Thời gian lưu trú (TT) = Thời gian ra khỏi hệ thống – Thời gian đến

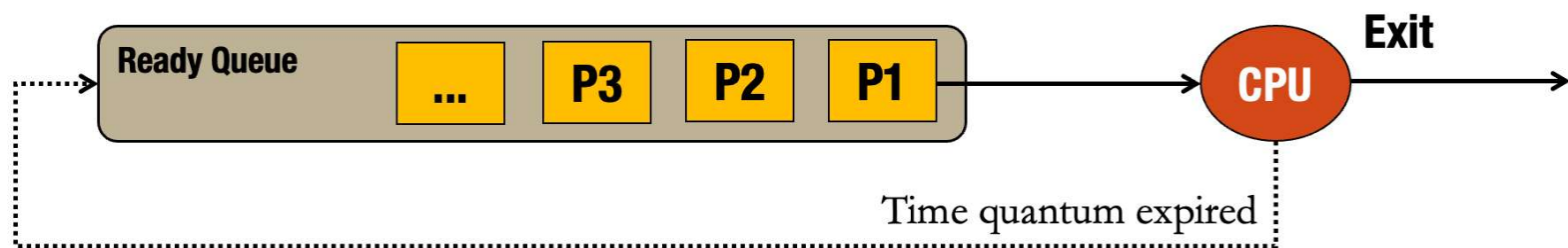
Thời gian chờ (WT) = TT – Thời gian thực thi (CPU Burst)

VD: $TT(P3) = 32 - 2 = 30$, $WT(P3) = 30 - 3 = 27$



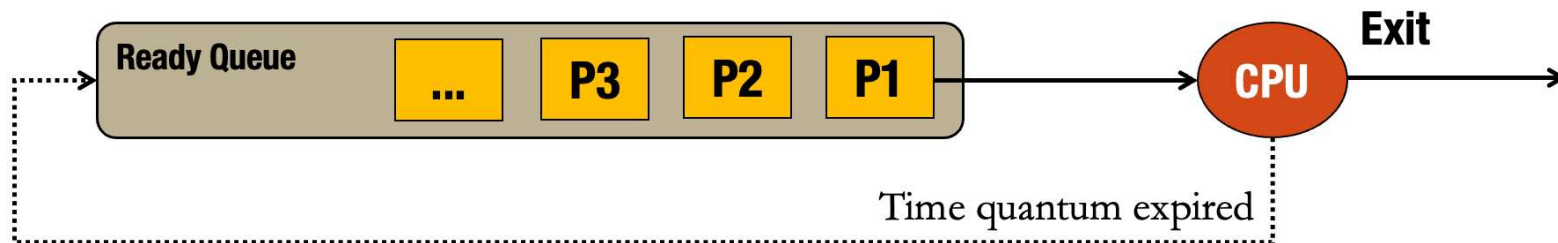
ĐIỀU PHỐI XOAY VÒNG (Round Robin)

- Không độc quyền
- Tiến trình được cấp một lượng tử thời gian q (time slice/time quantum), sau q hệ thống sẽ thu hồi CPU để chạy tiến trình kế tiếp theo thứ tự đến hệ thống của tiến trình.



ĐIỀU PHỐI XOAY VÒNG (Round Robin)

- 😊 Đơn giản, công bằng
- 😊 Giảm thời gian chờ của tiến trình ngắn
- 😊 Phù hợp hệ thống tương tác
- 😞 Chọn q bao nhiêu là tốt nhất?
- 😞 Chưa cân nhắc tiến trình cần ưu tiên



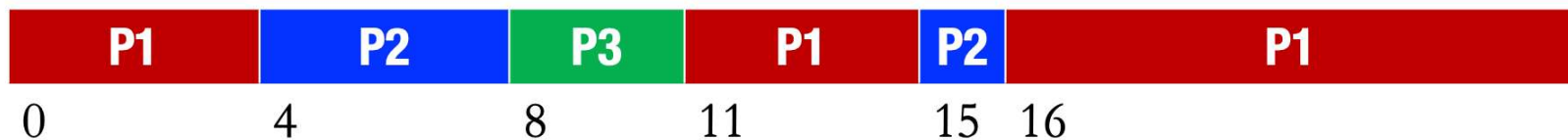
ĐIỀU PHỐI XOAY VÒNG (Round Robin)

Tiến trình	Thời gian đến	TG thực thi	TT	WT
P1	0	24	$32 - 0 = 32$	$32 - 24 = 8$
P2	1	5	$16 - 1 = 15$	$15 - 5 = 10$
P3	2	3	$11 - 2 = 9$	$9 - 3 = 6$
			AVG_{TT} = 18.66	AVG_{WT} = 8

$$WT(P1) = (11 - 4) + (16 - 15) = 7 + 1 = 8$$

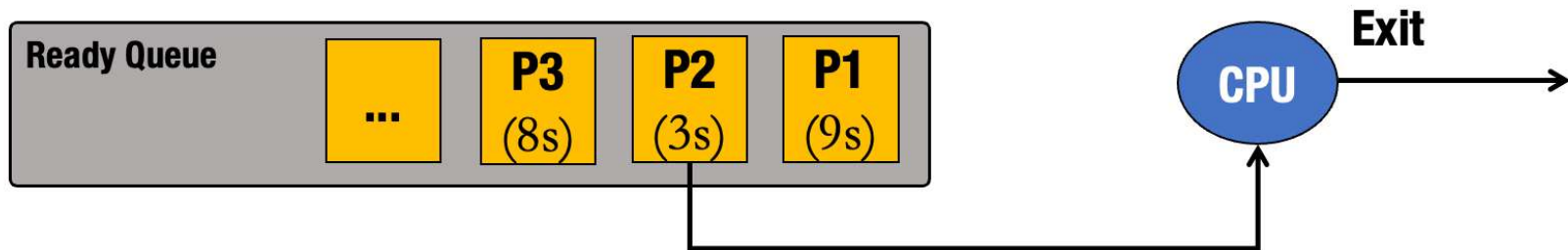
$$WT(P2) = (4 - 1) + (15 - 8) = 3 + 7 = 10; WT(P3) = (8 - 2) = 6$$

q = 4



ĐIỀU PHỐI SJF (Shortest Job First)

- Độc quyền
- Chọn tiến trình có thời gian thực thi (CPU Burst) ngắn nhất
- ☺ Đơn giản, cực đại hoá thông lượng
- ☹ Làm sao xác định CPU Burst?
- ☹ Vấn đề “starvation



ĐIỀU PHỐI SRTN (Shortest Remaining Time Next)

- Phiên bản không độc quyền của SJF
- Chọn tiến trình có thời gian CPU còn lại ngắn nhất.

Tiến trình	Thời gian đến	TG thực thi	TT	WT
P1	0	24	$32 - 0 = 32$	$32 - 24 = 8$
P2	1	5	$9 - 1 = 8$	$8 - 5 = 3$
P3	2	3	$5 - 2 = 3$	$3 - 3 = 0$
			AVG_{TT} = 14.33	AVG_{WT} = 3.66



ĐIỀU PHỐI VỚI ĐỘ ƯU TIÊN

- Có thể độc quyền hoặc không độc quyền.
- Mỗi tiến trình được cấp một độ ưu tiên và tiến trình có độ ưu tiên cao nhất sẽ được chọn thực thi.
- Phiên bản không độc quyền: CPU sẽ bị thu hồi nếu tiến trình có độ ưu tiên cao hơn đến hệ thống.
- 😊 Cân nhắc độ ưu tiên.
- ☹️ Vấn đề “starvation” → Giải pháp “aging”.



ĐIỀU PHỐI VỚI ĐỘ ƯU TIÊN

Giả sử giá trị ưu tiên càng lớn tiến trình càng được ưu tiên → $P3 > P2 > P1$.

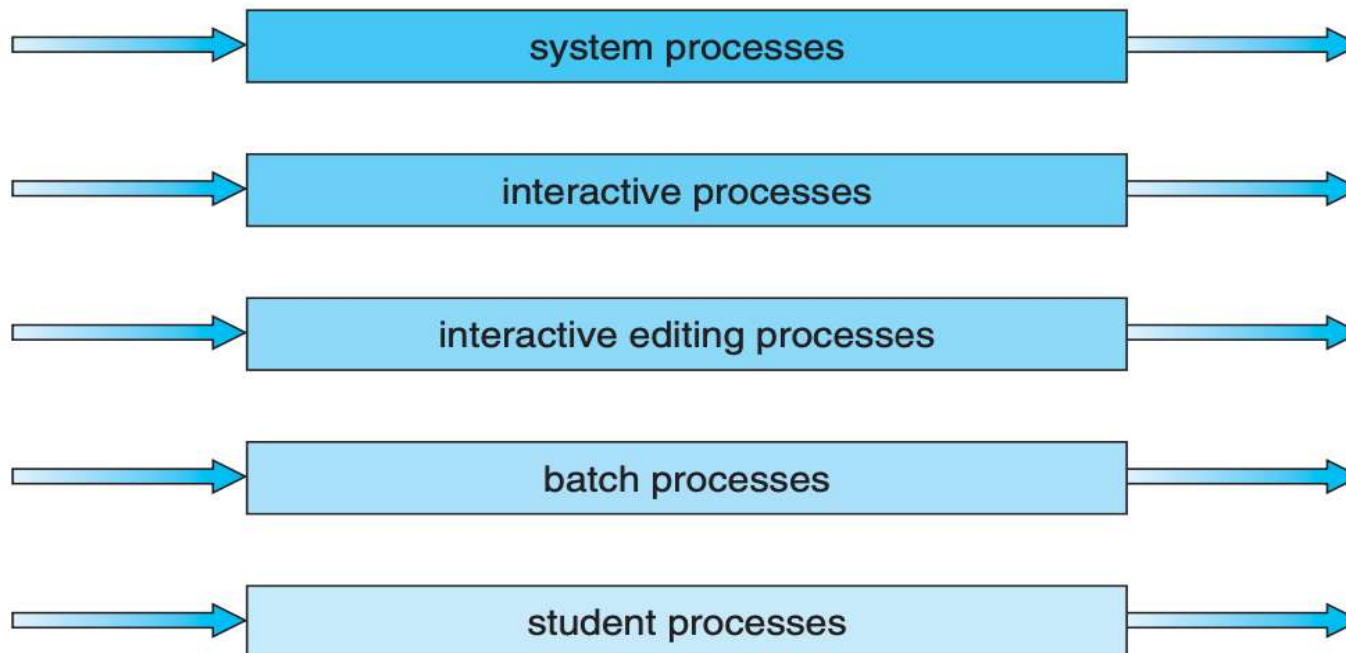
Tiến trình	Thời gian đến	Độ ưu tiên	TT	WT
P1	0	12	$32-0 = 32$	$32-24 = 8$
P2	1	18	$9-1 = 8$	$8-5 = 3$
P3	2	22	$5-2 = 3$	$3-3 = 0$
			AVG_{TT} = 14.33	AVG_{WT} = 3.66

Điều phối với độ ưu tiên không độc quyền



ĐIỀU PHỐI ĐA CẤP

highest priority



lowest priority

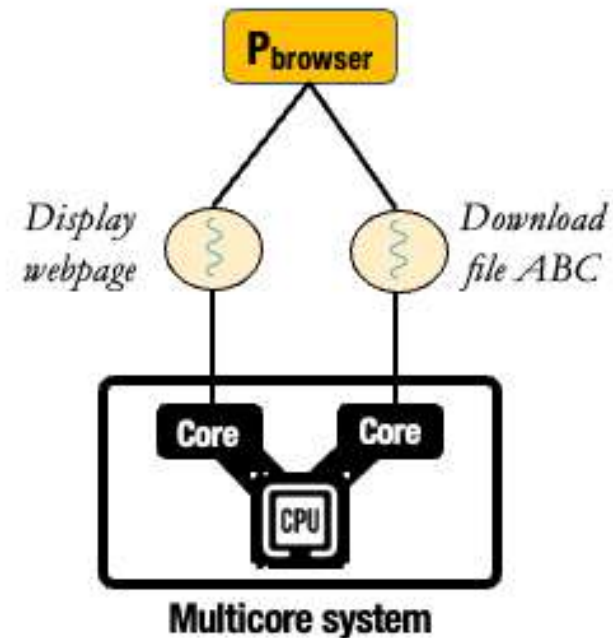
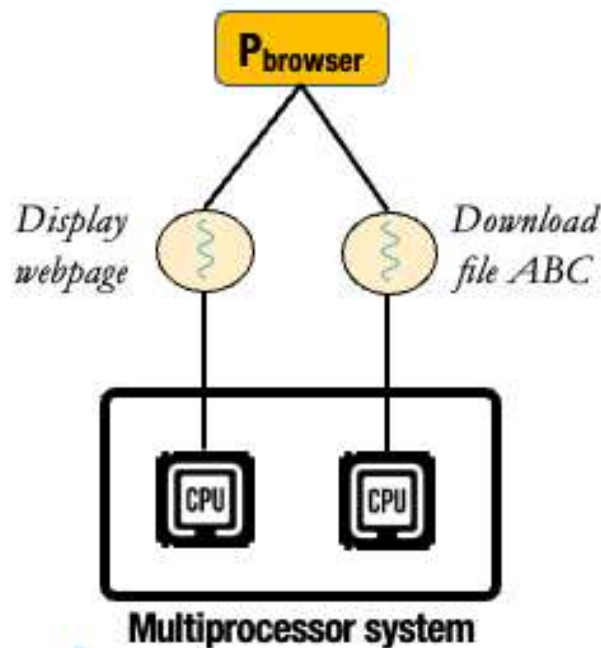


Nội dung

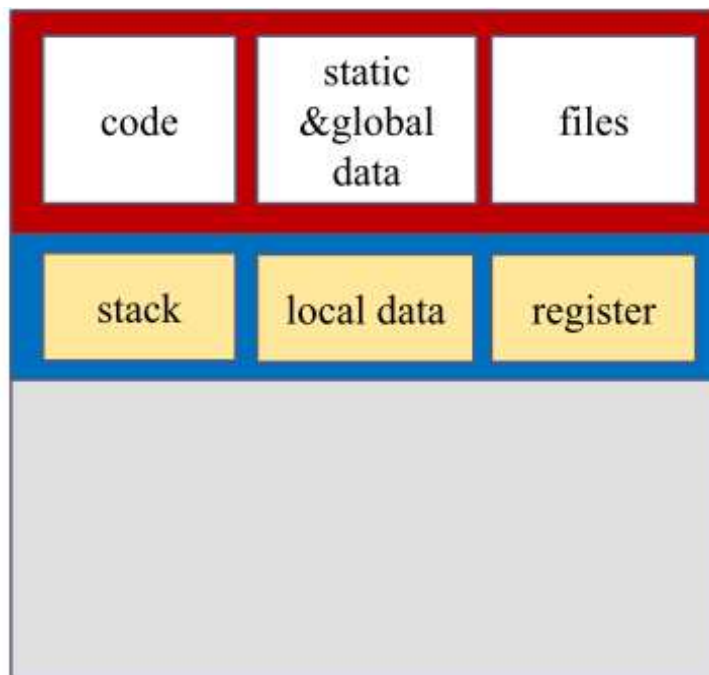
- Quản lý tiến trình
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”

KHÁI NIỆM TIỂU TRÌNH/LUỒNG

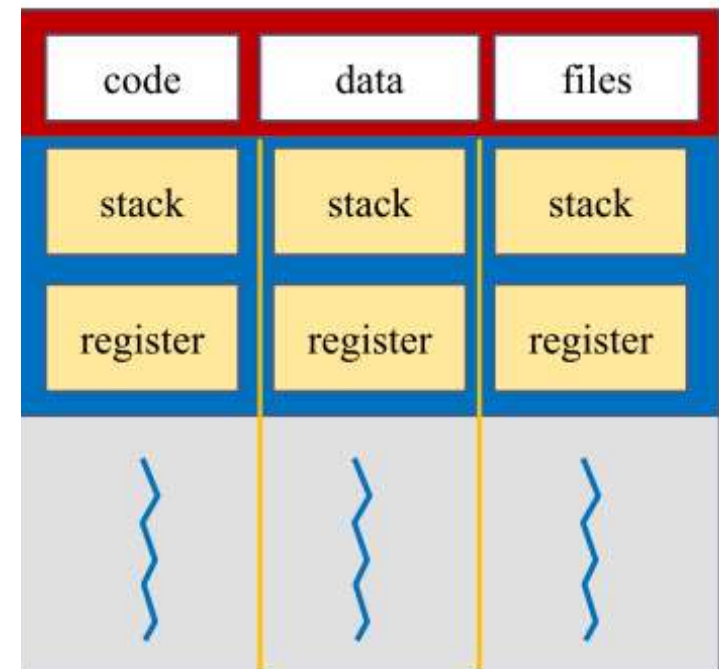
- Luồng thực thi bên trong một tiến trình.
- Có thể được thực thi song song, độc lập với các luồng khác.



KHÁI NIỆM TIỂU TRÌNH/LUỒNG (2)



Single-threaded process



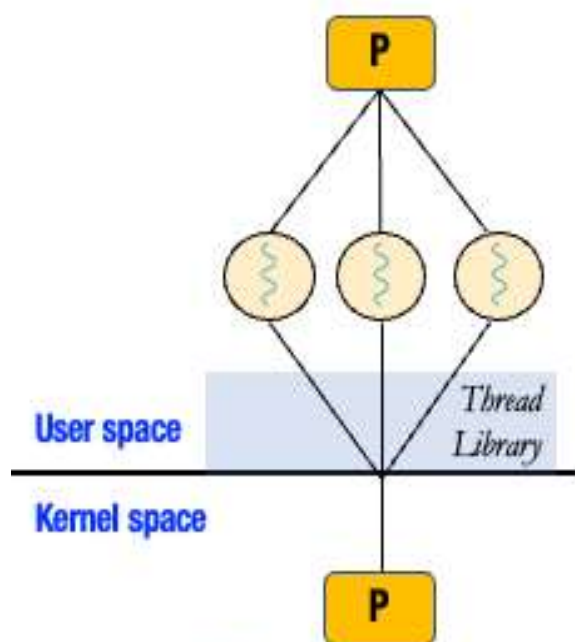
Multi-threaded process

ĐẶC TRƯNG TIỂU TRÌNH/LUỒNG

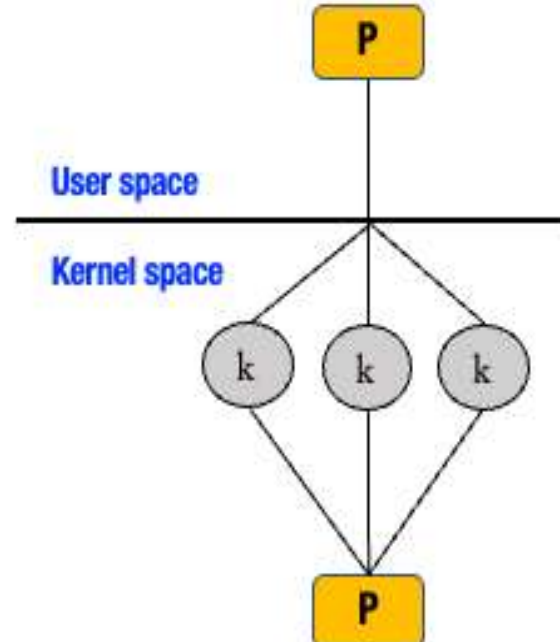
- Tính phản hồi nhanh.
- Chia sẻ tài nguyên dễ dàng.
- Tiết kiệm chi phí.
- Dễ dàng triển khai trên hệ thống đa CPU/đa nhân.



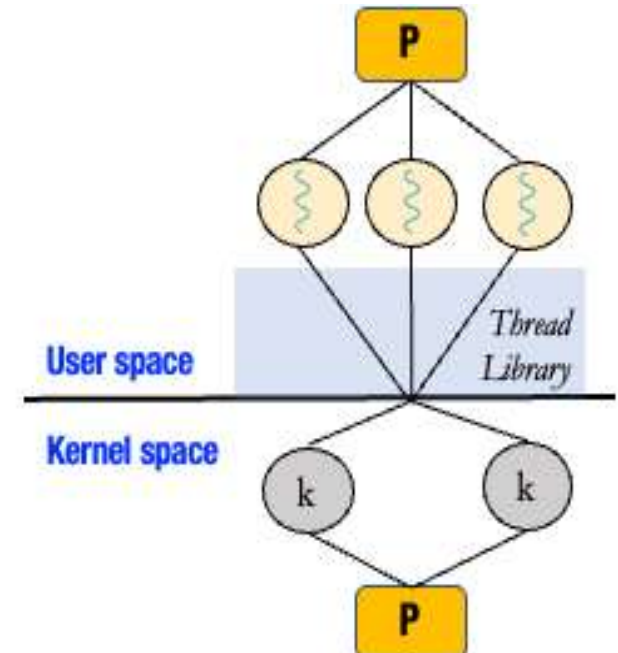
PHÂN LOẠI TIỂU TRÌNH



 **User-level thread**
Supported in user space
(Tiểu trình người dùng)

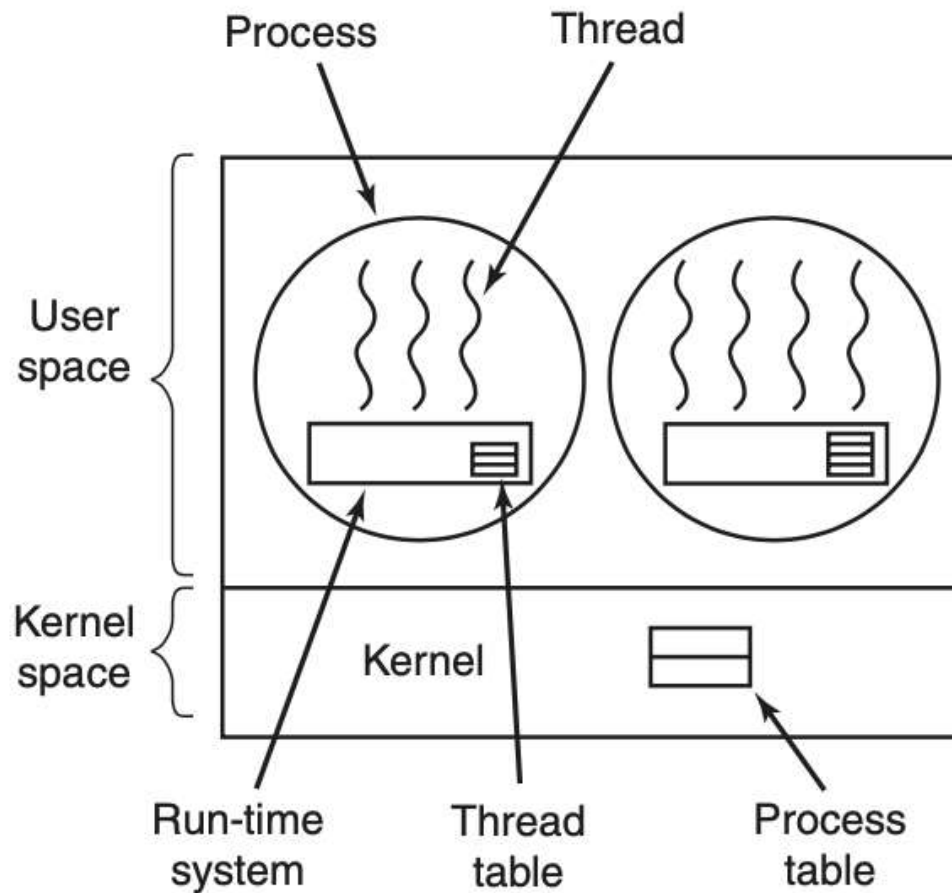


 **Kernel-level thread**
Supported in kernel space
(Tiểu trình hệ thống)

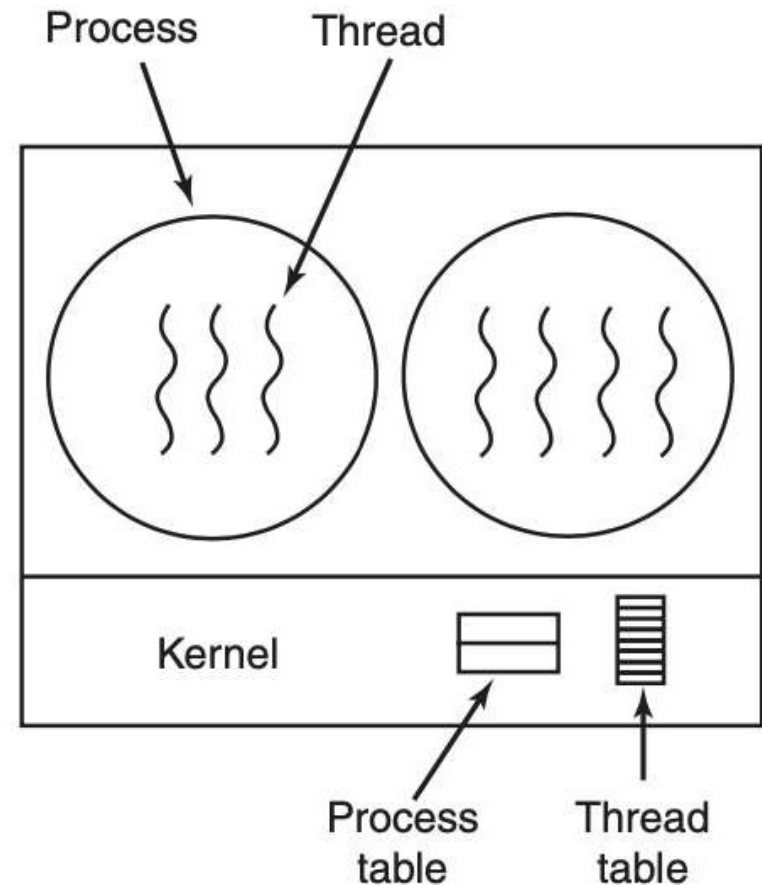


Hybrid thread
Supported in both user and kernel space

QUẢN LÝ TIỂU TRÌNH

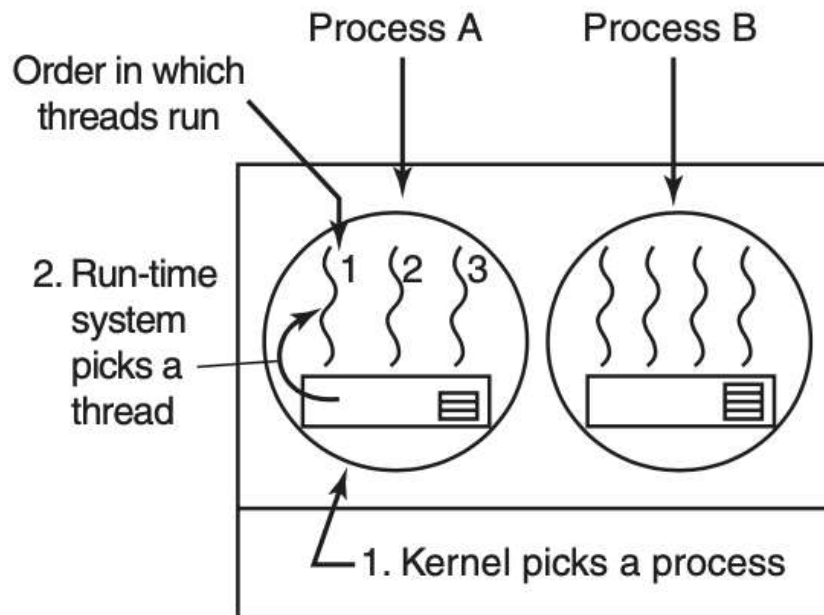


User-level thread



Kernel-level thread

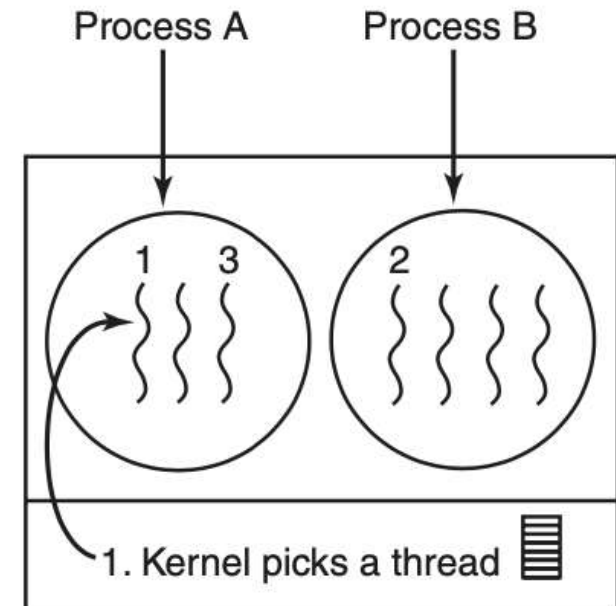
QUẢN LÝ TIỂU TRÌNH (2)



Possible: A1, A2, A3, A1, A2, A3

Not possible: A1, B1, A2, B2, A3, B3

User-level thread



Possible: A1, A2, A3, A1, A2, A3

Also possible: A1, B1, A2, B2, A3, B3

Kernel-level thread

Nội dung

- Quản lý tiến trình
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
 - **Giải pháp đồng bộ hoá**
- Vấn đề “bế tắc”

TÌNH HUỐNG TƯỞNG TRANH

```
Int availTicket = 1;
```

```
Client A ( ) {  
  1 if (availTicket > 0) {  
    4 availTicket -= 1;  
  }  
}
```

```
Client B ( ) {  
  2 if (availTicket > 0) {  
    3 availTicket -= 1;  
  }  
}
```

~~availTicket = -1~~

MIỀN GẮNG VÀ TÀI NGUYÊN GẮNG

- Miền găng (Critical Section): Đoạn chương trình có sử dụng biến hoặc tài nguyên dùng chung giữa các tiến trình.

```
Int availTicket = 1;
```

```
Client A ( ) {  
    if (availTicket > 0) {  
        availTicket -= 1;  
    }  
}
```

```
Client B ( ) {  
    if (availTicket > 0) {  
        availTicket -= 1;  
    }  
}
```

ĐIỀU KIỆN CỦA GIẢI PHÁP ĐỒNG BỘ

- **ĐK1:** Độc quyền truy xuất trên miền găng.
- **ĐK2:** Tiến trình ngoài miền găng không được khoá tiến trình muốn vào miền găng.
- **ĐK3:** Không có tiến trình nào phải đợi vô hạn.
- **ĐK4:** Không có sự giả lập về tốc độ tiến trình.



CÁC NHÓM GIẢI PHÁP ĐỒNG BỘ

Busy Waiting

```
while (KHÔNG CÓ QUYỀN VÀO MIỀN GẮNG);  
critical_section;
```

- **Giải pháp phần mềm**

- ✓ Biến cờ hiệu
- ✓ Luân phiên
- ✓ Peterson

- **Giải pháp phần cứng**

- ✓ Vô hiệu hoá ngắt
- ✓ Chỉ thị TSL

CÁC NHÓM GIẢI PHÁP ĐỒNG BỘ (2)

Sleep and Wakeup

```
if (KHÔNG CÓ QUYỀN VÀO MIỀN GẮNG)
    sleep();
critical_section;
wakeup(another);
```

- Semaphore
 - ✓ Semaphore nhị phân (mutex)
 - ✓ Semaphore đếm
- Monitor

BUSY WAITING: GIẢI PHÁP CỜ HIỆU

```
int lock = 0;
```

```
Process_A() {  
    while (1) {  
        while (lock);  
        lock = 1;  
        critical_section;  
        lock = 0;  
        ...  
    }  
}
```

```
Process_B() {  
    while (1) {  
        while (lock);  
        lock = 1;  
        critical_section;  
        lock = 0;  
        ...  
    }  
}
```

BUSY WAITING: GIẢI PHÁP LUÂN PHIÊN

```
int turn = 0;
```

```
Process_A() {  
    while (1) {  
        while (turn != 0);  
        critical_section;  
        turn = 1;  
        ...  
    }  
}
```

```
Process_B() {  
    while (1) {  
        while (turn != 1);  
        critical_section;  
        turn = 0;  
        ...  
    }  
}
```

BUSY WAITING: GIẢI PHÁP PETERSON

```
#define N 2          //two processes/threads solution
int turn;           //current turn
int interested[N];  //initial value = 0, N: number of threads
```

```
enterCS (int process) { //process = 0 or 1
    int other = 1 - process;
    interested[process] = 1;
    turn = other;
    while (turn == other &&
           interested[other] == 1); }
leaveCS (int process) {
    interested[process] = 0; }
```

```
Process_A() {
    while(1){
        enterCS(0);
        critical_section;
        leaveCS(0);
    }
}
```


BUSY WAITING: VÔ HIỆU HOÁ NGẮT

- Vô hiệu hoá ngắt (ngắt đồng hồ, ngắt từ thiết bị nhập/xuất, ...) khi vào miền găng và tái kích hoạt ngắt khi ra khỏi miền găng.
- Giải pháp chỉ dùng ở chế độ đặc quyền của HĐH.

Disable interrupts

Enter CS

critical_section

Leave CS

Enable interrupts

No interrupts

→ **No CPU Scheduling**

→ **No critical section problems**

BUSY WAITING: CHỈ THỊ TSL

```
Test_And_Set_Lock (bool lock) {  
    bool test = lock;  
    lock = true;  
    return test;  
}
```

Chỉ thị “nguyên tố”
không thể tách rời.

```
bool lock = false;  
Thread A {  
    while (Test_And_Set_Lock ( lock ));  
    critical_section;  
    lock = false;  
    return test;  
}
```

SLEEP & WAKEUP: SEMAPHORE

- **Biến số nguyên** (Integer variable) được sử dụng để hạn chế quyền truy cập vào vùng găng (Critical Section) thông qua hai thao tác nguyên tố: **down** và **up**
- **Down** (còn gọi là P hoặc wait)
 - Được gọi trước khi tiến vào vùng găng
 - Kiểm tra xem luồng gọi có được phép truy cập vùng găng không
- **Up** (còn gọi là V hoặc signal)
 - Được gọi khi ra khỏi vùng găng
 - Giải phóng vùng găng và đánh thức luồng đang chờ (nếu có)

SLEEP & WAKEUP: SEMAPHORE (2)

```
semaphore S = value;  
down(S)  
critical_section  
up(S)
```

```
typedef struct semaphore{  
    int value;  
    struct thread *list; //blocking  
    threads  
} semaphore;
```

```
down(semaphore *S) {  
    S→value--;  
    if (S→value < 0) {  
        add calling thread to S→list;  
        block();  
    }  
}
```

```
up(semaphore *S) {  
    S→value++;  
    if (S→value <= 0) {  
        remove a thread from S→list;  
        wakeup(A);  
    }  
}
```

SLEEP & WAKEUP: SEMAPHORE (3)

- Binary semaphore (Semaphore nhị phân)
 - Tương tự như cơ chế khóa (còn gọi là mutex lock)
 - Giá trị: 0 hoặc 1
- Counting semaphore (Semaphore đếm)
 - Dùng để kiểm soát quyền truy cập vào tài nguyên có số lượng hữu hạn
 - Được khởi tạo với số lượng tài nguyên có sẵn

SLEEP & WAKEUP: SEMAPHORE (4)

```
semaphore mutex = 1;  
processA () {  
    down(mutex);  
    critical_section  
    up(mutex); }  
}
```

```
semaphore count = 5;  
processX () {  
    down(count);  
    critical_section  
    up(count); }  
}
```

```
semaphore mutex = 0; // processB → processA  
processA () {  
    down(mutex);  
    ...  
}  
processB () {  
    ...  
    up(mutex);  
}
```

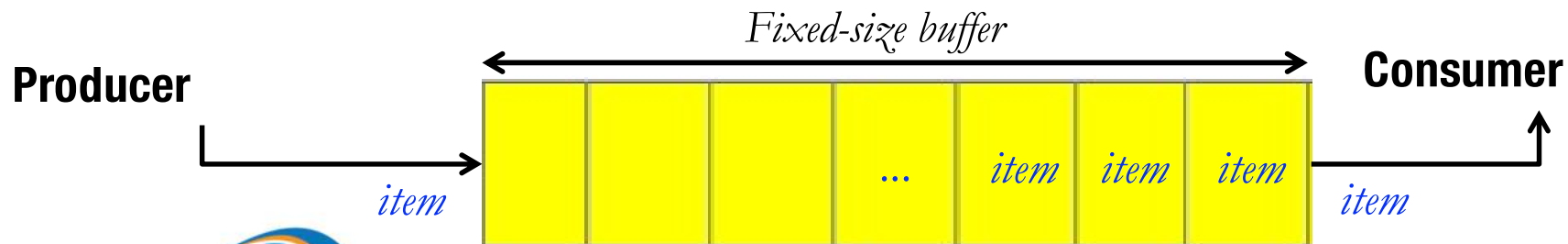
Nội dung

- Quản lý tiến trình
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
 - **Các bài toán đồng bộ hóa kinh điển**
- Vấn đề “bế tắc”

NHÀ SẢN XUẤT - NGƯỜI TIÊU THỤ

Producer – Consumer Problem

- Vấn đề bộ đệm có giới hạn
 - Nhà sản xuất tạo ra dữ liệu và đặt vào bộ đệm dùng chung.
 - Người tiêu dùng lấy dữ liệu từ bộ đệm và tiêu thụ.
 - Nếu bộ đệm đầy → nhà sản xuất bị chặn.
 - Nếu bộ đệm rỗng → người tiêu dùng bị chặn.
 - Chỉ một tiến trình được truy cập bộ đệm tại một thời điểm.



NHÀ SẢN XUẤT - NGƯỜI TIÊU THỤ (2)

Producer – Consumer Problem

```
#define N 50  
semaphore mutex, empty, full;  
init (mutex, 1), init (empty, 0), init (full, N);
```

```
producer() {  
    down(full);  
    down(mutex);  
    insertItem(item);  
    up(mutex);  
    up(empty);  
}
```

```
consumer() {  
    down(empty);  
    down(mutex);  
    item = removeItem()  
    up(mutex);  
    up(full);  
}
```



NHÀ SẢN XUẤT - NGƯỜI TIÊU THỤ (3)

Producer – Consumer Problem

```
#define N 50  
semaphore mutex, empty, full;  
init (mutex, 1), init (empty, 0), init (full, N);
```

```
producer() {  
    down(mutex);  
    down(full);  
    insertItem(item);  
    up(mutex);  
    up(empty);  
}
```



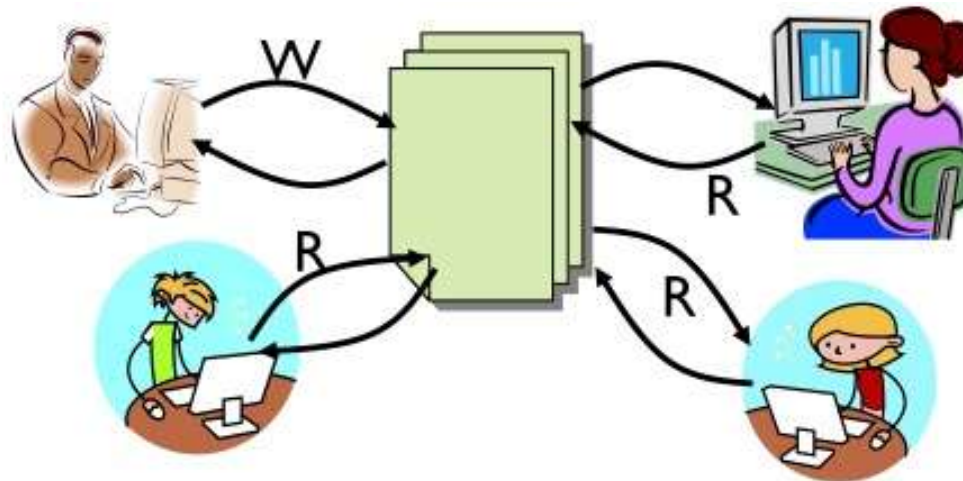
```
consumer() {  
    down(empty);  
    down(mutex);  
    item = removeItem();  
    up(mutex);  
    up(full);  
}
```

ĐỌC – GHI DỮ LIỆU

Readers – Writers Problem

○ Vấn đề đọc ghi trên cơ sở dữ liệu

- Nhiều người đọc có thể đọc cùng lúc từ cơ sở dữ liệu.
- Nếu có người ghi đang ghi → các luồng khác phải đợi.
- Nếu có ít nhất một người đang đọc → người ghi phải đợi.



ĐỌC – GHI DỮ LIỆU (2)

Readers – Writers Problem

```
semaphore db, mutex;  
init (db, 1), init (mutex, 1);  
int nReaders = 0;
```

```
Writer ( ) {  
    down(db);  
    writeToDB( );  
    up(db);  
}
```

```
Reader ( ) {  
    down(mutex);  
    if (nReaders==0) down(db);  
    nReaders+=1;  
  
    readFromDB( );  
  
    nReaders-=1;  
    if (nReaders==0) up(db);  
    up(mutex);  
}
```



ĐỌC – GHI DỮ LIỆU (3)

Readers – Writers Problem

```
semaphore db, mutex;  
init (db, 1), init (mutex, 1);  
int nReaders = 0;
```

```
Writer ( ) {  
  down(db);  
  writeToDB( );  
  up(db);  
}
```

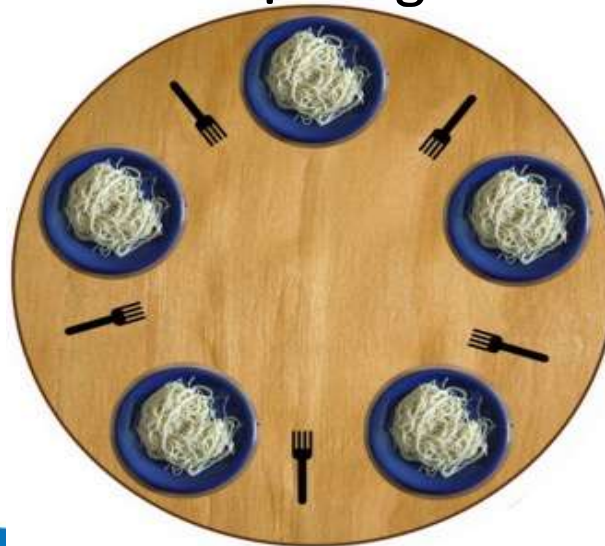
```
Reader ( ) {  
  down(mutex);  
  if (nReaders==0) down(db);  
  nReaders+=1;  
  up(mutex);  
  readFromDB( );  
  down(mutex);  
  nReaders-=1;  
  if (nReaders==0) up(db);  
  up(mutex);  
}
```



BỮA ĂN TỐI CỦA CÁC TRIẾT GIA

Dining Philosophers Problem

- Vấn đề chia sẻ tài nguyên
 - 5 triết gia ngồi quanh bàn tròn, có 5 chiếc nĩa.
 - Mỗi triết gia cần 2 cái nĩa gần nhất để ăn.
 - Chỉ được nhặt một nĩa tại một thời điểm.
 - Không được nhặt nĩa đã bị hàng xóm lấy.



BỮA ĂN TỐI CỦA CÁC TRIẾT GIA (2)

Dining Philosophers Problem

```
semaphore forks[5]; //forks[1, 1, 1, 1, 1]
```

```
Philosopher (int i) { //i: philosopher number
    while(true) {
        down(forks[i]); //take left fork
        down(forks[(i + 1)%5]); //take right fork
        eating( );
        up(forks[i]); //put left fork back
        up(forks[(i + 1)%5]); //put right fork back
    }
}
```

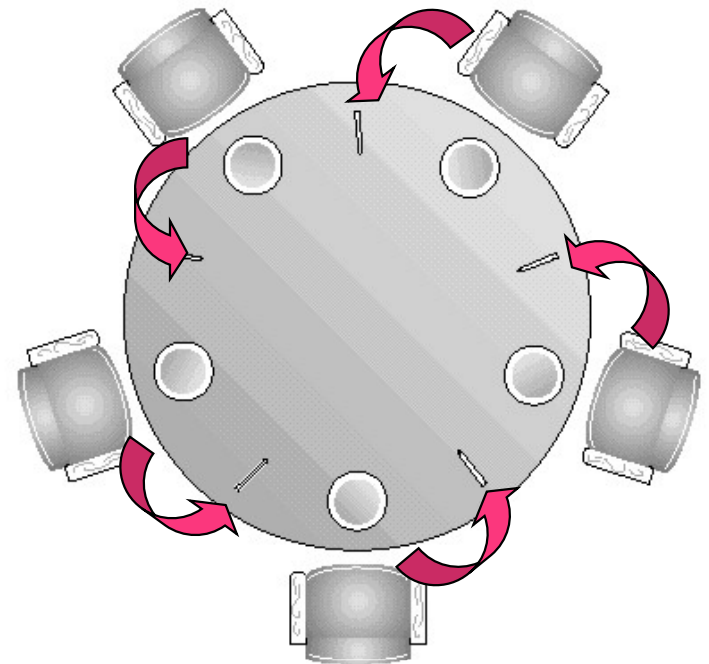
**Deadlock**

Nội dung

- Quản lý tiến trình
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”
 - **Khái niệm**

ĐỊNH NGHĨA

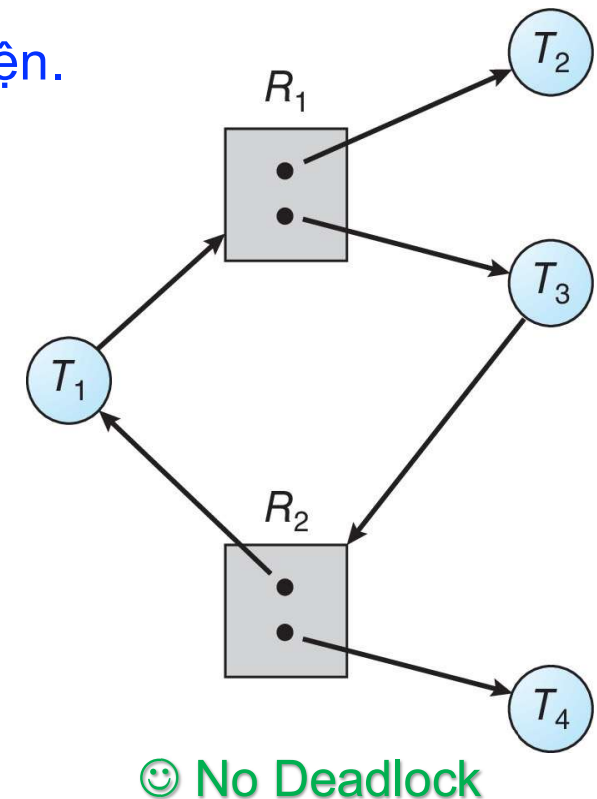
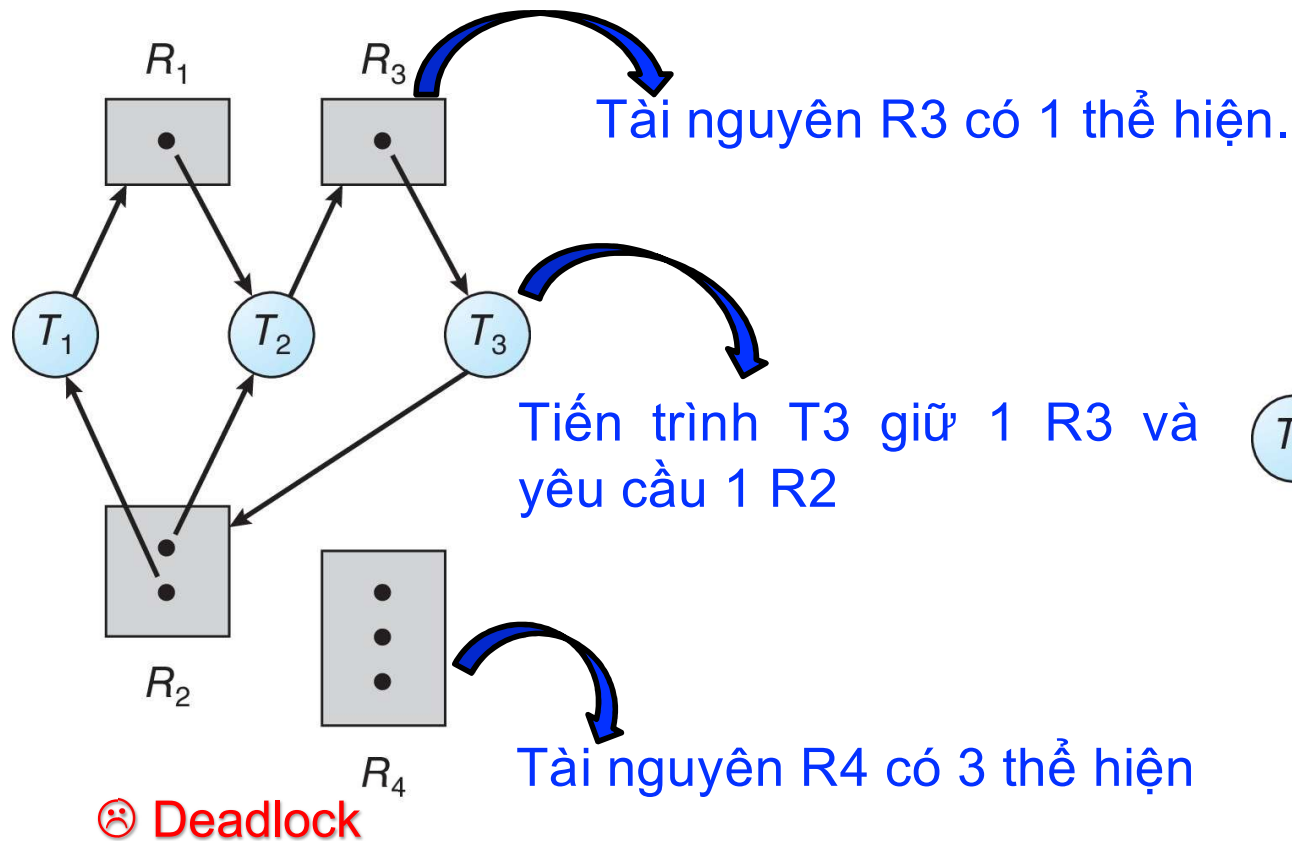
Bế tắc hay tắc nghẽn (deadlock) là trạng thái trong đó hai hay nhiều tiến trình tranh chấp tài nguyên và không tiến trình nào có thể kết thúc để giải phóng tài nguyên.



☹ Deadlock

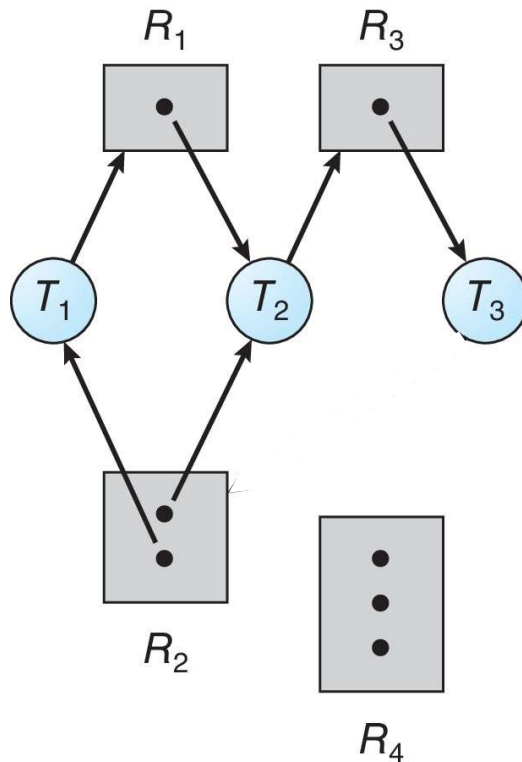
ĐỒ THỊ CẤP PHÁT TÀI NGUYÊN

Resource-Allocation Graph (RAG)



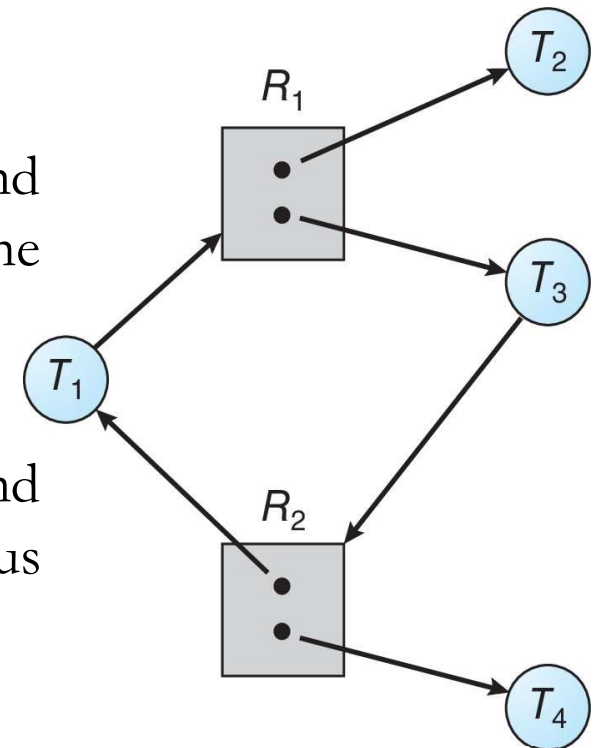
ĐỒ THỊ CẤP PHÁT TÀI NGUYÊN (2)

Resource-Allocation Graph (RAG)



☺ No Deadlock

- Graph with no cycle
→ *no deadlock*
- Graph with a cycle and resources having only one instance
→ *deadlock*
- Graph with a cycle and resources having various instances
→ *deadlock possibility*



☺ No Deadlock

ĐIỀU KIỆN XẢY RA BẾ TẮC

- Hệ thống xuất hiện bế tắc khi **bốn điều kiện sau đồng thời xảy ra:**
 - Độc quyền (Mutual Exclusion).
 - Giữ và đợi (Hold and Wait).
 - Không chiếm quyền (No Preemption).
 - Chờ đợi vòng tròn (Circular Waiting).

Nội dung

- Quản lý tiến trình
- Điều phối CPU
- Quản lý tiểu trình/luồng
- Đồng bộ hoá
- Vấn đề “bế tắc”
 - **Giải pháp**

GIẢI PHÁP

- #1: Bỏ qua, không xử lý (Ostrich Algorithm).
- #2: Ngăn chặn điều kiện xảy ra bế tắc (Prevention).
- #3: Phòng tránh (Avoidance).
- #4: Phát hiện và khôi phục (Detection and Recovery).



#1 – KHÔNG XỬ LÝ



*Deadlock?
I don't care 😊*

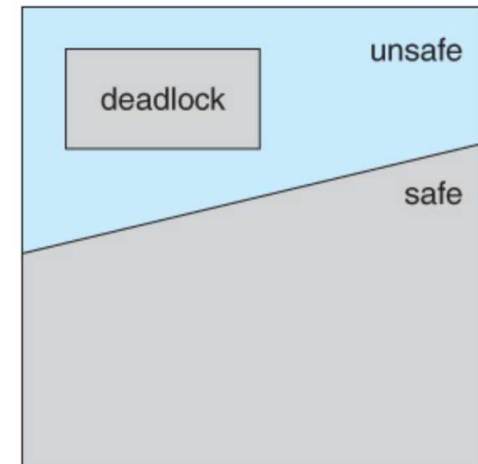
#2 – NGĂN CHẶN ĐIỀU KIỆN BẾ TẮC

- **Độc quyền:** không thể ngăn chặn.
- **Giữ-đợi & Chờ đợi vòng tròn:**
 - Tiến trình y/c tất cả tài nguyên một lúc và chỉ được cấp khi toàn bộ tài nguyên được đáp ứng.
 - Tiến trình phải trả tài nguyên cấp thấp thì mới được y/c tài nguyên cấp cao hơn.
- **Không chiếm quyền:**
 - Tiến trình đang giữ tài nguyên khi y/c tài nguyên khác mà không được cấp phát cần trả lại tài nguyên đang nắm giữ.

#3 – PHÒNG TRÁNH

- **Giải thuật nhà băng:** y/c thông tin bổ sung về các tài nguyên tiến trình đòi hỏi và kiểm tra xem việc cấp phát tài nguyên có dẫn tới hệ thống “không an toàn” → Trì hoãn cấp phát.

- **Allocation:** Ma trận cấp phát
- **Max:** Ma trận khai báo số lượng tối đa.
- **Need:** Ma trận y/c tài nguyên còn lại.
 - $\text{Need} = \text{Max} - \text{Allocation}$
- **Available:** Ma trận tài nguyên sẵn sàng.



#3 – PHÒNG TRÁNH

	Allocation			Max			Need		
	C	D	E	C	D	E	C	D	E
P ₀	0	1	0	7	5	3	7	4	3
P ₁	2	0	0	3	2	2	1	2	2
P ₂	3	0	2	9	0	2	6	0	0
P ₃	2	1	1	2	2	2	0	1	1
P ₄	0	0	2	4	3	3	4	3	1



Hệ thống có an toàn?

Một chuỗi an toàn khả thi:
(P₁, P₃,

Available (Work)		
C	D	E
3	3	2

#3 – PHÒNG TRÁNH

	Allocation			Max			Need		
	C	D	E	C	D	E	C	D	E
P ₀	0	1	0	7	5	3	7	4	3
P ₁	2	0	0	3	2	2	1	2	2
P ₂	3	0	2	9	0	2	6	0	0
P ₃	2	1	1	2	2	2	0	1	1
P ₄	0	0	2	4	3	3	4	3	1



Hệ thống có an toàn?

Một chuỗi an toàn khả thi:
(P₁, P₃,

Available (Work)		
C	D	E
7	4	3

#3 – PHÒNG TRÁNH

	Allocation			Max			Need			
	C	D	E	C	D	E	C	D	E	
P ₀	0	1	0	7	5	3	7	4	3	✓
P ₁	2	0	0	3	2	2	1	2	2	✓
P ₂	3	0	2	9	0	2	6	0	0	✓
P ₃	2	1	1	2	2	2	0	1	1	✓
P ₄	0	0	2	4	3	3	4	3	1	✓

Hệ thống có an toàn? ✓

Một chuỗi an toàn khả thi:
(P₁, P₃, P₀, P₂, P₄)

Available (Work)		
C	D	E
10	5	7

#4 – PHÁT HIỆN VÀ KHÔI PHỤC

- **Phát hiện:** Thuật toán phát hiện bế tắc
 - **Allocation:** Ma trận cấp phát
 - **Request:** Ma trận yêu cầu hiện tại
 - **Available:** Ma trận tài nguyên sẵn sàng.
 - **Finish:** Ma trận trạng thái kết thúc (1: có thể kết thúc).
- **Khôi phục**
 - Kết thúc tiến trình trong chuỗi tiến trình bị bế tắc
 - Quay lui hệ thống về trạng thái an toàn trước đó
 - Thu hồi tài nguyên phá vỡ chu trình

#4 – PHÁT HIỆN VÀ KHÔI PHỤC

	Allocation			Request (Claim)		
	C	D	E	C	D	E
P₀	0	1	0	7	5	3
P₁	2	0	0	3	2	2
P₂	3	0	2	9	0	2
P₃	2	1	1	2	2	2
P₄	0	0	2	4	3	3



Available		
C	D	E
7	4	5

☹ Deadlock

Finish				
P₀	P₁	P₂	P₃	P₄
0	1	0	1	1

Tài liệu tham khảo

- Giáo trình hệ điều hành, Trần Trung Dũng, Phạm Tuấn Sơn, 2021, NXB Khoa Học Kỹ Thuật.
- Modern Operating System, 4th Edition, Andrew Tanenbaum & Herbert Bos, 2014, Pearson.
- Operating System Concepts, 10th Edition, Abraham Silberschatz, Peter B. Galvin & Greg Gagne, 2018, Wiley.